MDPI

Article

A Comparative Evaluation of SPARQ Against Leading Metaheuristics

Vasileios Charilogis[1], Ioannis G. Tsoulos[2,*] and Gianni Anna Maria[3]

[1] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, v.charilog@uoi.gr
[2] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, itsoulos@uoi.gr
[3] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, am.gianni@uoi.gr
* Correspondence: itsoulos@uoi.gr

Abstract: SPARQ is an advanced global optimization algorithm, the direct evolution of an earlier method called ARQ2. It searches efficiently for the best solution to problems where the space of options is too vast to check exhaustively, such as tuning antenna arrays or planning fuel-efficient spacecraft trajectories. Like its predecessor, it works with a population of candidate solutions that improve generation after generation, alternating between two complementary search strategies. What sets SPARQ apart is that it makes nearly every part of this process adaptive. Its population shrinks intelligently as the search matures. Its internal settings draw from a memory of many past successful configurations, not a single average. Its escape-from-stagnation mechanisms come in graduated strength, from a gentle nudge to a deeper partial restart. It also adds capabilities its predecessor never had, including a dedicated phase that locally polishes the current best solution using its own memory of productive directions. Every addition is kept only where it showed an overall benefit during development, though a subsequent component-wise analysis shows this benefit varies markedly in size and statistical significance across mechanisms. The result is more reliable than its predecessor on the large majority of tested problems, while staying grounded in the same battle-tested core.

Keywords: SPARQ; Continuous optimization; Evolutionary algorithm; Adaptive parameter control; Metaheuristics; Differential evolution; Algorithm benchmarking; Population-based search;

1. Introduction

Optimization problems in which the goal is to locate the best possible configuration of a set of continuous variables, without relying on gradient information and without any guarantee that the objective function is smooth, convex, or even continuous, arise throughout engineering, physics, and applied economics. Formally, such problems are instances of global optimization:

$$\min_{\mathbf{x}\in\Omega} f(\mathbf{x}), \qquad \Omega := [\ell_1, u_1] \times \cdots \times [\ell_D, u_D] \subset \mathbb{R}^D,$$

where $f : \Omega \to \mathbb{R}$ is accessed only through pointwise evaluations, and derivative information may be unavailable. Given an evaluation budget N_{fe}, an algorithm produces a sequence of iterates $\{\mathbf{x}_t\}_{t=1}^{N_{fe}}$ and tracks the best-so-far value

$$f_{\min}(t) := \min_{1\le s\le t} f(\mathbf{x}_s)$$

The task is to locate the global minimum (or maximum) of f over Ω despite the presence of numerous local optima, ill-conditioning, and, in many real applications, a strict limit N_{fe} on how many times the objective function can be evaluated. The difficulty of this

Citation: Charilogis, V., Tsoulos, I.G., Gianni, A. M. A Comparative Evaluation of SPARQ Against Leading Metaheuristics. *Journal Not Specified* **2024**, *1*, 0.

Received:
Revised:
Accepted:
Published:

task grows sharply with the dimension D, and no single search strategy can be expected to perform best on every possible problem instance [1].

In response, the optimization literature has developed several broad families of methods. Exact and gradient-based methods offer strong convergence guarantees but require differentiability assumptions that most real-world objective functions do not satisfy. Classical local random search methods, most notably the algorithm of Solis and Wets [2], instead perturb a single candidate solution and accept improving moves, offering simplicity at the cost of a tendency to stall in local optima. To escape this limitation, the field turned to population-based metaheuristics, which explore the search space with many candidate solutions simultaneously. Early and influential examples include simulated annealing [3], genetic algorithms [4], together with more recent refinements to their initialization strategies such as k-means-based seeding [10,11], particle swarm optimization [7], and the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [8], later extended to large-scale problems via limited-memory variants such as LM-CMA-ES [9]. More recently, a growing number of bio-inspired metaheuristics have been proposed and subsequently refined through hybridization with local search and modern termination criteria, as in the case of the Giant-Armadillo Optimization method [5,6].

Among population-based metaheuristics, Differential Evolution (DE), introduced by Storn and Price [12] and since surveyed extensively [13], has proven especially effective and has spawned an active line of successive refinements. JADE [14] introduced the current-to-pbest mutation strategy together with an external archive of recently displaced solutions, both of which became standard components of later variants. SHADE [15] replaced JADE's single adaptive parameter pair with a historical memory of successful settings, and L-SHADE [16] combined this with a linear population size reduction scheme building on earlier work on population sizing [17]. jSO [18] refined this further with progress-dependent parameter schedules, and subsequent work introduced rank-based selective pressure in choosing difference vectors [19], culminating in NL-SHADE-RSP [20], which combined non-linear population size reduction with rank-based selection and adaptive archiving. A parallel line of research explored hybridizing DE with other search paradigms and enriching its crossover operator: CoBiDE [21] introduced covariance-guided, bimodal parameter sampling, individual-dependent DE variants (IDE) proposed alternative generation-based scheduling [22], eigenvector-based ("eigen") crossover, aligning the crossover operation with the population's natural axes of variation rather than the raw coordinate axes, was introduced in [23] and later integrated into a cooperative multi-algorithm framework, EA4Eig, that combines CoBiDE, IDE, CMA-ES, and jSO under a shared eigen-crossover mechanism [24]. Separately, other general-purpose mechanisms have been proposed for escaping stagnation and premature convergence, including opposition-based learning [25], [26], Lévy-flight perturbations based on the numerical simulation method of Mantegna [27], and, from the domain of sequential decision-making, Thompson sampling as a principled method for adaptively allocating trials between competing strategies [28].

The method presented in this article, SPARQ, is the latest member of a family of Differential-Evolution-based optimizers developed internally, beginning with ARQ [29] and its direct successor ARQ2 [30], both of which combine a current-to-pbest DE mutation strategy with an alternative individual-dependent evolution strategy (IDE) selected via an adaptive roulette mechanism, restricted tournament replacement, an external archive, and outlier quarantine. SPARQ builds directly on ARQ2's architecture while replacing nearly every one of its fixed or singly-adapted mechanisms with an adaptive counterpart drawn from the DE literature surveyed above, among them a SHADE-style historical memory in place of a single running parameter average, non-linear population size reduction, rank-based selective pressure in the style of NL-SHADE-RSP, a jSO-style progress-dependent schedule for its elitist pressure, eigen-coordinate crossover, a Thompson-sampling bandit in place of frequency-based strategy selection, and Lévy-flight-based quarantine perturbations. On top of this synthesis, SPARQ introduces two mechanisms with no counterpart in ARQ2 or the algorithms above: a stagnation-gated local polishing phase based on the classical

Solis-Wets adaptive random search, and a short-term memory of recently successful search directions ("trajectory echo") that is replayed during this local phase.

The remainder of this article is organized as follows. Section 2 describes the architecture inherited from ARQ2 in detail. Section 3 presents each of SPARQ's adaptive and novel mechanisms in turn, together with the motivation behind each. Section 4 describes the experimental configuration and reports and discusses the results, comparing SPARQ against nine other state-of-the-art optimizers on the standard-dimensionality benchmark and against five large-scale optimizers on the high-dimensional benchmark, together with an analysis of computational scalability with dimension. Section 5 concludes.

2. Background: The ARQ and ARQ2 Predecessor Methods

2.1. ARQ: A Cohesive Optimization Design with Outlier Quarantine

ARQ [29] is a differential-evolution-based optimizer built around a single, cohesive update cycle that combines four complementary mechanisms.

Archive-assisted pbest/1/bin trial generation. Each new candidate is generated by attracting a target vector toward one of the top-p fraction of the population, plus a difference term drawn partly from the live population and partly from an external archive of recently displaced solutions:

$$v = x + F\,(x_{pbest} - x) + F\,(r_1 - r_2), \qquad r_1 \in P_t,\ r_2 \in P_t \cup A_t.$$

This is followed by binomial crossover and projection back onto the feasible box.

Success-history parameter adaptation. The scale factor F and crossover rate CR are drawn from a Cauchy and a Normal distribution, respectively, centered on running means μ_F, μ_{CR}. After each cycle, these means are updated using gain-weighted statistics, a weighted Lehmer mean for F and a weighted arithmetic mean for CR, so that parameter settings that produced larger fitness gains are trusted more.

Restricted Tournament Replacement (RTR). A trial vector first competes against its own parent, if it fails to improve, it is instead compared against its nearest neighbor (in bounds-normalized distance) within a small random pool drawn from the population, and replaces that neighbor if superior. This localizes selection pressure and helps preserve population diversity.

Outlier quarantine and targeted micro-restart. ARQ's central contribution is an event-driven mechanism that detects fitness outliers using a robust interquartile-range threshold $\theta = Q3 + a \cdot IQR$ and gently relocates a fraction of them toward a robust center (the mean of the best half of the population) under a small perturbation, accepting the repair only if it improves fitness. A complementary micro-restart, triggered after a fixed number of non-improving iterations, perturbs a small fraction of the worst individuals around the current incumbent. Together, these two mechanisms convert destabilizing outliers into controlled exploration stimuli, narrowing the gap between best-case and average-case performance without sacrificing peak results.

2.2. ARQ2: A Roulette-Guided Hybrid Extension of ARQ

ARQ2 [30] preserves this entire stability-oriented backbone but extends it into an asymmetric hybrid architecture in which two internal search branches, ARQ and an auxiliary IDE (Individual-dependent Differential Evolution) branch, operate on a shared population and a shared external archive.

Roulette-based branch scheduling. At each iteration, a controller selects which of the two branches to execute, based on branch "credits" that are updated in proportion to each branch's recent number of successful replacements. Branches with higher recent credit are selected more often, but a reset mechanism restores both branches to a common baseline credit whenever either branch's selection probability falls below a threshold δ, preventing one branch from permanently dominating and collapsing the hybrid into a single-strategy method. An initial bootstrap phase enforces the ARQ branch for a fixed number of early iterations, before branch competition becomes informative.

The IDE branch. When selected, this branch constructs its donor vectors using its own, per-individual memories of F and CR, and a progress-dependent elite-guided pool whose size and guidance strength shift over the course of the search, favoring stronger elite guidance early on, and a more balanced elite/random mixture later. It further switches between two donor forms (guidance-based and current-based) depending on a phase threshold that adapts if the branch's success rate stalls.

An enriched ARQ branch. When the ARQ branch is selected, its mutation rule is enriched with a jSO-style, progress-dependent attraction coefficient $K(\pi)$ that modulates how strongly each new candidate is pulled toward an elite exemplar, growing more elitist as the search progresses.

Asymmetric stabilization. Importantly, the quarantine and micro-restart mechanisms inherited from ARQ remain attached only to the ARQ branch. This preserves the original design's corrective role without duplicating the same machinery inside the auxiliary IDE branch, while still letting IDE contribute a structurally distinct search dynamic that helps on multimodal landscapes.

Empirically, this hybridization was shown to primarily strengthen repeated-run reliability: across 36 benchmark problems, ARQ2 improved on ARQ's mean-value performance on 21 problems (matching it on 11, and trailing on only 4), while remaining highly competitive in best-case terms [30].

3. The Proposed Method: SPARQ

3.1. The mechanisms of the SPARQ

SPARQ is built directly on top of ARQ2's architecture (Section 2.2): it retains the shared population, the shared external archive, and the two-branch structure (ARQ vs. IDE), but it replaces nearly every one of ARQ2's fixed or singly-adapted mechanisms with an adaptive counterpart, and adds several mechanisms with no counterpart in ARQ2 at all. This section describes each addition in turn.

Population State and Non-Linear Population Size Reduction (NLPSR)

Unlike ARQ2, which runs with a fixed population size N throughout, SPARQ starts from an initial size N_{init} (by default scaled with dimension, $N_{init} = 18D$) and shrinks it toward a floor N_{min} as the search progresses, following

$$N(t) = \text{round}\Big(N_{init} + (N_{min} - N_{init}) \cdot p(t)^{e(t)}\Big), \qquad e(t) = 1 - (1 - \alpha_{\text{NL}}) \cdot p(t),$$

where $p(t) \in [0, 1]$ is the fraction of the evaluation budget consumed so far and $\alpha_{\text{NL}} \in (0, 1]$ controls the shape of the reduction: $\alpha_{\text{NL}} = 1$ recovers a purely linear LSHADE-style schedule, while $\alpha_{\text{NL}} < 1$produces a near-linear reduction early in the search that accelerates as the budget is exhausted, in the spirit of NL-SHADE-RSP. At the end of every iteration, if the current population exceeds the target size, the worst individuals are discarded, freeing up evaluations for the remaining ones. The cached eigenbasis (Section 3.1) is invalidated whenever the population shrinks, since it must be recomputed on the new, smaller sample.

For the comparative evaluation reported in Section 4, however, N_{init} was fixed at 100 for SPARQ and for every other population-based competitor in the comparison, overriding the dimension-scaled default $N_{init} = 18D$. This choice ensures that all compared methods start from an identical population size rather than from method-specific, dimension-dependent values, so that observed performance differences reflect the search mechanisms themselves rather than differences in initial population sizing. The dimension-scaled schedule $N_{init} = 18D$ remains SPARQ's default, recommended setting outside of this controlled comparison.

SHADE-Style Historical Memory for F and CR

ARQ2 keeps a single, continuously-updated pair of running means (μ_F, μ_{CR}). SPARQ instead maintains a circular memory of size H, $\{(M_F^k, M_{CR}^k)\}_{k=1}^{H}$, one slot of which (the "terminal" slot, following jSO) is permanently pinned to a high value ($M_F^H = M_{CR}^H = 0.9$)

rather than being overwritten. Before generating each trial, a memory slot r is drawn uniformly at random, and

$$CR \sim \mathcal{N}(M_{CR}^r, 0.1), \qquad F \sim \text{Cauchy}(M_F^r, 0.1),$$

both resampled/clamped to their valid ranges ($CR \in [0,1] CR \in [0,1] CR \in [0,1], F \in [F_{lo}, F_{hi}]$). Two jSO-style schedules further constrain these draws early in the search: CR is floored at 0.7 before 20% progress and at 0.6 before 50% progress, and F is capped at $0.7 + 0.3\, p(t)$ before 60% progress. After each generation, the (non-terminal) memory slots are updated cyclically using the same gain-weighted Lehmer/arithmetic mean rule as ARQ2, but only one slot advances per generation rather than the single running pair, so that a diverse set of previously-successful settings remains available for future sampling rather than being collapsed into one blended average.

Rank-Based Selective Pressure and jSO-Style Elitism Schedule

ARQ2 draws the difference-vector companions r_1 (and, when not drawn from the archive, r_2) uniformly at random from the population. SPARQ instead biases this choice toward better-ranked individuals: given the population sorted by fitness (rank 0 = best), an index at rank r is selected with probability proportional to $(N - r)^{k_r}$, where the selective-pressure exponent itself increases linearly over the run,

$$k_r(t) = k_{r,init} + (k_{r,final} - k_{r,init}) \cdot p(t), \qquad k_{r,init} = 2,\ k_{r,final} = 3,$$

so that rank bias is moderate early on and sharpens later, in the style of LSHADE-RSP. Independently, the size of the $p$best pool from which the elite-attraction target x_{pbest} is drawn also narrows over the course of the run, following a jSO-style linear schedule from $p_{max} = 0.25$ down to $p_{min} = 0.10$, so that the elitist pull becomes progressively more selective as the search matures. A further NL-SHADE-RSP-style refinement, CR-sorting, reassigns the sampled CR values within each generation's batch of parents so that better-ranked parents receive the smallest values (conservative, fine-grained crossover) and worse-ranked parents the largest (more disruptive recombination).

Eigen-Coordinate Binomial Crossover

ARQ2's binomial crossover operates coordinate-by-coordinate in the raw problem coordinates, which is ill-suited to correlated, non-separable landscapes whose productive directions run diagonally across several variables at once. SPARQ periodically estimates the dominant axes of the current population's spread: using the best 50% of individuals (by fitness), it computes their empirical covariance matrix, diagonalizes it via Jacobi rotation to obtain an eigenbasis B and per-axis scale factors, and refreshes this basis every `eig_period` iterations (or whenever the population is resorted or resized). With probability $p_{eig} = 0.40$, and whenever D stays within a practical cap ($D \leq 100$, since the Jacobi decomposition is $\mathcal{O}(D^3)$ per sweep), the standard binomial crossover inherited from ARQ2's formulation, under which $u_{ij} = v_{ij}$ whenever $\text{rand}_{ij} \leq CR_i$ or $j = j_{rand}$, and $u_{ij} = x_{ij}$ otherwise, is instead applied in the rotated coordinate system: both the parent and the mutant are projected onto the eigenbasis ($x_e = B^\top x$), crossover is performed there, and the result is rotated back ($u = B\, u_e$). This lets the operator recombine along the population's natural axes of variation rather than the arbitrary problem axes.

Thompson Sampling Bandit for Strategy Selection

ARQ2 selects between the ARQ and IDE branches using a frequency-based "credit" roulette with a threshold-triggered reset. SPARQ replaces this with a genuine two-armed Bayesian bandit: each branch k maintains a Beta posterior with parameters (a_k, b_k), initialized with a mild prior favoring ARQ ($a_{ARQ} = 2$) to provide brief warm-up bias before a short deterministic bootstrap (a small fixed number of forced ARQ iterations) hands control to the bandit. At each iteration, a sample is drawn from each arm's posterior via two Gamma draws, and the arm with the larger sample is selected. After a branch executes, its posterior is updated in proportion to its observed success rate, normalized by the number of trial evaluations actually attempted so that a large IDE sweep and a smaller ARQ sweep

are weighted comparably, between iterations, both posteriors are geometrically decayed toward their uninformative prior (1,1) to accommodate a non-stationary environment, in which a strategy that was previously effective may cease to be so as the population and progress evolve.

Lévy-Flight Quarantine

ARQ2's quarantine mechanism relocates fitness outliers (detected via the interquartile threshold $\theta = Q_3 + \alpha \cdot IQR$) toward the centroid of the top half of the population using an ordinary Gaussian perturbation. SPARQ instead perturbs each repaired outlier using a symmetric Lévy-stable step, generated via the Mantegna (1994) algorithm: a step $u/|v|^{1/\beta}$ is formed from two correlated Gaussian draws u, v with $\beta = 1.5$, producing a distribution that is mostly small but occasionally very large, mimicking the search pattern of animals foraging under scarce resources. Applied per coordinate and scaled by the box range, this gives quarantined individuals a genuine chance of relocating to a distant, unexplored region rather than merely oscillating near the population centroid.

On-Demand Opposition-Based Basin Escape

This mechanism has no counterpart in ARQ2. It monitors two independent signals: the number of consecutive iterations without any improvement in the incumbent best ($no\_improve \geq stag\_trigger$), and the population's normalized spread (the average, per-coordinate, box-normalized standard deviation across the population) falling below a small collapse ratio. When both conditions hold simultaneously, and a cooldown counter has elapsed, a fraction of the worst individuals are relocated to points computed as a quasi-opposition of their current position, either the exact reflection through the center of the box, or a point interpolated between that reflection and the current incumbent best, and the relocation is accepted only if it improves fitness. This directly targets premature convergence: rather than waiting for a general-purpose restart, it intervenes as soon as the population has visibly collapsed onto a single region while genuinely failing to progress, forcing a look at the opposite side of the search space.

Stagnation-Gated Elite Polish

This is the most substantial addition with no analogue in ARQ2. Whenever the incumbent best has failed to improve by a meaningful relative margin for a number of iterations (this trigger tightens to a single stagnant iteration in the closing quarter of the budget), SPARQ enters a short, budget-capped burst (never more than a fixed fraction, by default 12%, of the evaluations consumed so far) of local refinement attempts centered on the current best x_{best}. Each attempt draws one of four probe modes at random:

1. Single-coordinate probing (round-robin over dimensions, so that every coordinate is eventually visited even in high dimension): either a coordinate-wise opposition ($x_j \to \ell_j + u_j - x_j$) or an adaptive Gaussian perturbation of that one coordinate
2. Anisotropic eigen-aligned steps, drawn as a Gaussian in the (possibly slightly stale) eigenbasis of Section 3.4, scaled per-axis by the square root of the corresponding eigenvalue, so that steps are long along the population's natural "valley" and short across it
3. Trajectory echo (described in Section 3.1)
4. Solis-Wets adaptive random search (described in Section 3.1 below, integrated as the fourth probe mode).

Each successful probe (one that improves $best_f$) is injected into the population by overwriting its current worst member, propagating the refinement into the DE core's own mutation pool, probe-mode-specific step sizes are adapted by a 1/5-success rule (grown on success, shrunk on failure). The whole mechanism is protected by three independent safety nets: a hard evaluation-budget cap, an exponential cooldown backoff whenever an entire burst fails to find any improvement, and an efficiency arbitration that compares the gain-per-evaluation achieved by the polish against the gain-per-evaluation achieved by the ordinary evolutionary loop over the same interval, forcing the polish to stand down whenever the core DE search is demonstrably the better investment of evaluations. A landscape-level self-selection rule additionally imposes a long cooldown whenever several

consecutive bursts produce only negligible relative gains, so that the mechanism naturally becomes inactive on landscapes (such as those with a distant global optimum reachable only through large exploratory jumps) where local refinement of the current incumbent is not productive.

Trajectory Echo: A Memory of Recently Successful Directions

Also with no counterpart in ARQ2: whenever the DE core (either the ARQ branch's restricted-tournament acceptance or the IDE branch's greedy replacement) accepts a strictly improving trial, the accepted displacement (trial vector minus the individual it replaced) is pushed into a small, fixed-size ring buffer of at most eight entries. As one of the elite-polish probe modes (Section 3.1), a candidate step is formed as a random linear combination of the buffered directions, normalized so that its expected magnitude stays comparable to a single one of its ingredients regardless of how many are currently buffered. This lets the polish phase replay an echo of movements that have already proven productive during this specific run, of particular value on problems where genuine improvement requires shifting several coordinates together in a coordinated way, a pattern that isotropic or single-coordinate probing cannot easily discover. Like the other probe modes, this mechanism carries its own circuit breaker: a run of repeated failures disables it for the remainder of the search rather than continuing to spend evaluations on a direction source that has stopped paying off.

Solis-Wets Adaptive Local Search

The fourth elite-polish probe mode is a single running trajectory, based on the classical adaptive random search method of Solis and Wets (1981), anchored at the current incumbent. A bias vector b (initialized at zero) accumulates a memory of productive directions. Each attempt draws a random perturbation δ and first tries $x_{best} + b + \delta$, if that fails to improve, it tries the exact reflection $x_{best} - b - \delta$ using the same random draw before giving up. A success in the biased direction grows b toward that direction and expands the step scale, a success only in the reflected direction shrinks b and partially reverses it, and a failure in both directions decays b toward zero and contracts the step, the classical Solis-Wets rule for avoiding wasted wandering. Because this probe mode only ever evolves a single trajectory from the current incumbent, rather than recombining information from two different population members, it cannot produce the kind of structurally incoherent points that a population-level recombination operator could produce on permutation-symmetric landscapes.

Hard-Stagnation Rejuvenation

ARQ2's single micro-restart mechanism is replaced in SPARQ by a two-tier system. A survival trigger fires whenever the population's fitness spread has collapsed to near-zero (a looser condition than any typical similarity-based stopping rule), and re-seeds only a small sliver (about 5%) of the population uniformly at 3.1random, at each iteration if needed, its purpose is purely to keep the population from appearing fully converged to an external stopping rule while genuine progress is still possible. A hard-stagnation trigger fires only once the incumbent has failed to improve for a much longer interval (a configurable multiple of the basin-escape threshold of Section 3.1) and the population has also collapsed, when it fires, it re-seeds a much larger fraction of the population (by default, all but the top 25%) uniformly at random, while explicitly preserving the incumbent best itself, which is never part of the population and is therefore never at risk. An escalation mechanism further tracks whether repeated firings of either tier keep failing to produce meaningful gains: a persistent survival-tier failure forces a hard-stagnation escape early, and a persistent hard-stagnation failure escalates to a near-complete reset (keeping only a small safety fraction), preventing a strong-but-insufficient partial restart from being immediately "reconquered" by an entrenched elite on unusually deceptive landscapes. The SHADE memory of Section 3.1 is deliberately preserved rather than reset across a rejuvenation event, since it continues to encode useful parameter statistics for the refreshed population.

Overall Control Flow

Each SPARQ iteration proceeds as follows. The archive is trimmed to its capacity, a branch (ARQ or IDE) is selected, either by the initial bootstrap or by the Thompson bandit of Section 3.1, and executed, with the ARQ branch additionally followed by Lévy-flight quarantine (Section 3.1). Two independent stagnation counters are then updated, one tracking any improvement at all (feeding the basin-escape and rejuvenation triggers), and one tracking only meaningful relative improvement (feeding the elite-polish trigger, so that large populations at high dimensionality, where some individual improves the incumbent by a vanishing amount almost every generation, do not starve the polish mechanism of activations). The elite polish (Section 3.1), the basin escape (Section 3.1), and the rejuvenation mechanism (Section 3.1) are then each given the opportunity to fire, in that order, subject to their own independent cooldowns and triggers. The population is then shrunk toward its current NLPSR target size if needed (Section 3.1), and the bandit posteriors are decayed before the next iteration begins.

3.2. Algorithmic Summary

Algorithm 1 Main workflow of SPARQ

INPUT
- f: objective function
- $\Omega \subset \mathbb{R}^D$: box-constrained search domain, dimension D
- N_{init}, $N_{\min}$, α_{NL}: NLPSR: initial/minimum population size and shrink-shape exponent
- T_{eval}: maximum number of function evaluations
- H, M_F, M_{CR}, F_{lo}, F_{hi}: SHADE memory size, circular memories, and F bounds
- $p_{\max}$, $p_{\min}$: jSO-style pbest fraction schedule
- k_r^0, k_r^1: LSHADE-RSP rank-selection pressure schedule
- p_{eig}, T_{eig}, ρ_{eig}: eigen-crossover probability, refresh period, and top-fraction
- α_A: archive-rate coefficient
- $\{(a_k, b_k)\}_{k=1}^2$, γ_b, n_{bs}: Thompson bandit priors (ARQ/IDE), decay rate, bootstrap length
- α_q, ρ_q, β_L, σ_L: Levy quarantine: IQR coefficient, repair fraction, Levy exponent, step scale
- τ_s, ε_v, ρ_b, κ_b: OBL basin escape: stagnation trigger, spread-collapse ratio, fraction, cooldown
- τ_p, π_p, ρ_p, B_p: elite polish: trigger, late-phase threshold, burst fraction, budget cap
- m_r, κ_r, C_r: rejuvenation: stagnation factor, keep-fraction, cooldown

OUTPUT
- x^*, f^*

INITIALIZATION
- Set $N \leftarrow N_{\text{init}}$, initialize population $X = \{x_i\}_{i=1}^N \subset \Omega$ and evaluate all individuals
- Set archive $A \leftarrow \varnothing$
- Identify the incumbent best pair (x^*, f^*)
- Set $f^{\text{prev}} \leftarrow f^*$, $s \leftarrow 0$, $s_p \leftarrow 0$
- Initialize the SHADE memory (M_F, M_{CR}) with the terminal slot fixed at 0.9
- Initialize the bandit state $(a_1, b_1) \leftarrow (2, 1)$, $(a_2, b_2) \leftarrow (1, 1)$, and the bootstrap counter n_{bs}
- Initialize the individual IDE memories $\{F_i^{\text{IDE}}, CR_i^{\text{IDE}}\}_{i=1}^N$
- Invalidate the eigen-basis and set $e \leftarrow N$

SPARQ main pseudocode

```
01 While e < T_eval do
02     Trim A to size ⌊α_A N⌋
03     If n_bs > 0 then
04         Select the ARQ branch; set n_bs ← n_bs − 1
05     Else
06         Sample branch k via Thompson sampling from {(a_k, b_k)}_{k=1}^2
07     Endif
08     If k = IDE then
09         Apply the IDE branch update (Algorithm 2)
10     Else
11         Refresh the eigen-basis B if stale (Algorithm 3)
12         Apply the ARQ branch update to X and A (Algorithm 4)
13         Apply the Levy-flight quarantine correction using α_q, ρ_q, β_L, σ_L (5)
14     Endif
15     Update the bandit posterior (a_k, b_k) for branch k
16     If f* < f^prev then
17         Set f^prev ← f*, s ← 0
18     Else
19         Set s ← s + 1
20     Endif
21     Update the polish-stagnation counter s_p from the relative gain in f*
22     If s_p ≥ τ_p then apply the stagnation-gated elite polish (Algorithm 6)
23     If s ≥ τ_s and the population spread has collapsed then apply the OBL basin escape (Algorithm 7)
24     Apply the hard-stagnation rejuvenation check (Algorithm 8)
25     Set N to the current NLPSR target size and shrink X if needed (Algorithm 9)
26     Decay the bandit posteriors (a_k, b_k) toward the uniform prior
27     Trim A again to size ⌊α_A N⌋
28     Update the stopping state and report the current incumbent
29     Set e equal to the current number of function evaluations
30 Endwhile
31 Return x*, f*
```

Concretely, each pass through Algorithm 1 maps to the following steps. The archive is trimmed at the start of the loop (line 2). Branch selection happens in lines 3–7: a forced ARQ warm-up while the bootstrap counter $n_{bs} > 0$ (lines 3–4), after which the Thompson bandit samples a branch from $\{(a_k, b_k)\}$ (lines 5–6). The chosen branch is then executed in lines 8–14: the IDE update (Algorithm 2) if $k =$ IDE (line 9), or, on the ARQ side, an eigen-basis refresh if stale (Algorithm 3, line 11), the ARQ trial generation and RTR selection

(Algorithm 4, line 12), and the Lévy-flight quarantine correction (Algorithm (5, line 13). The bandit posterior for the executed branch is updated immediately afterward (line 15). Two independent stagnation signals are then maintained: the ordinary counter s, reset on any improvement and incremented otherwise (lines 16–20), and the meaningful-gain counter s_p (line 21), which gates the elite polish (Algorithm 6) once $s_p \geq \tau_p$ (line 22). The remaining stabilization mechanisms fire in sequence and are each individually gated: the OBL basin escape (Algorithm 7) once $s \geq \tau_s$ and the population spread has collapsed (line 23), and the hard-stagnation rejuvenation check (Algorithm 8), which evaluates its own survival/hard-stagnation conditions internally (line 24). The population is then shrunk toward its current NLPSR target size if needed (Algorithm 9, line 25), the bandit posteriors are decayed toward their uninformative prior (line 26), and the archive is trimmed a second time before the stopping state is updated and the current incumbent reported (lines 27–28).

Algorithm 2 IDE branch update in SPARQ

INPUT
- $X = \{x_i\}_{i=1}^{N}$: current population
- f, Ω: objective function and feasible domain
- g, $g_{\max}$, g_t, Λ, λ_c: IDE progress index, horizon, phase threshold, patience window, low-success counter
- $\{F_i^{\mathrm{IDE}}, CR_i^{\mathrm{IDE}}\}_{i=1}^{N}$: individual IDE parameter memories

OUTPUT
- Updated X, incumbent pair (x^*, f^*), updated g_t, λ_c

IDE branch update
01 Advance g (synced to global progress or incremented), clamped to $g_{\max}$
02 Sort X in nondecreasing objective value
03 Set $\varepsilon(g) \leftarrow 0.1 + 0.9 \cdot 10^{5(g/g_{\max}-1)}$
04 If $g < g_t$ then set $S_{th} \leftarrow 0$; otherwise set $S_{th} \leftarrow 0.1$
05 For each $i = 1, \ldots, N$ do
06 Sample distinct indices $o, r_1, r_2, r_3 \neq i$
07 Set $m(g) \leftarrow \max\{2, \mathrm{round}(\varepsilon(g)N)\}$
08 If $g \leq g_t$ then with probability $0.9\,\varepsilon(g)$ choose $r_1^\star$ from the best $m(g)$ individuals; otherwise $r_1^\star \leftarrow r_1$
09 Else with probability 0.5 choose $r_1^\star$ from the best $m(g)$ individuals; otherwise $r_1^\star \leftarrow r_1$
10 If $g > g_t$ and a Bernoulli(0.5) trial succeeds then
11 Set $v_i \leftarrow x_i + F_i^{\mathrm{IDE}}(x_{r_1^\star} - x_o) + F_i^{\mathrm{IDE}}(x_{r_2} - x_{r_3})$
12 Else
13 Set $v_i \leftarrow x_o + F_i^{\mathrm{IDE}}(x_{r_1^\star} - x_o) + F_i^{\mathrm{IDE}}(x_{r_2} - x_{r_3})$
14 Endif
15 Repair v_i to Ω
16 Construct y_i by binomial crossover of x_i and v_i with rate CR_i^{IDE}
17 Repair y_i to Ω, evaluate $f(y_i)$
18 Endfor
19 Define the success set $I_s = \{i : f(y_i) < f(x_i)\}$ and set $SR \leftarrow |I_s|/N$
20 If $g < g_t$ then
21 If $SR \leq S_{th}$ then $\lambda_c \leftarrow \lambda_c + 1$; otherwise $\lambda_c \leftarrow 0$
22 If $\lambda_c \geq \Lambda$ then $g_t \leftarrow g$
23 Endif
24 For each $i \in I_s$ do record the accepted step $y_i - x_i$ (trajectory echo), replace $x_i \leftarrow y_i$, and update (x^*, f^*) if improved
25 Return the updated X, (x^*, f^*), g_t, λ_c

Each individual is updated using a progress-dependent mix of guidance from an elite pool and random companions (lines 1–17), with the elite-guidance probability and pool size shrinking as the search matures. Successful trials replace their parents and feed the trajectory-echo memory (lines 19, 24), while a separate success-rate check can advance the branch's internal phase threshold early if progress stalls (lines 20–23).

Algorithm 3 Eigen-basis refresh in SPARQ

INPUT
- X: population, sorted by fitness
- D, N, ρ_{eig}, $D_{\min}$: dimension, population size, top-fraction, minimum dimension for activation

OUTPUT
- B (eigenvectors), $\mathbf{s}$ (per-axis scales), validity flag

Eigen-basis refresh
01 If $D > 100$ then mark the basis invalid and return
02 Set $top \leftarrow \max(D+2, \text{round}(\rho_{\text{eig}} N))$, clamped to $top \leq N$
03 If $D < D_{\min}$ or $top < \max(D+2, 4)$ then mark the basis invalid and return
04 Compute the mean $\bar{x}$ of the top top individuals
05 Compute the empirical covariance matrix C of the top top individuals about $\bar{x}$
06 Regularize the diagonal of C to avoid singularity
07 Diagonalize C via Jacobi rotation to obtain eigenvectors B and eigenvalues w
08 Set the per-axis scale $s_j \leftarrow \text{clip}(\sqrt{w_j / w_{\max}},\ 0.05,\ 1)$ for each axis j
09 Mark the basis valid and return B, $\mathbf{s}$

The covariance matrix of the best half of the population is computed and diagonalized (lines 4–7) to obtain the natural axes along which the population is currently spread. These axes, together with per-axis scale factors, are reused by both the ARQ crossover (Algorithm 4) and the elite polish (Algorithm 6) until the next refresh.

Algorithm 4 ARQ branch trial generation and RTR selection in SPARQ

INPUT
- X, A, N, D, Ω: population, archive, size, dimension, domain
- H, M_F, M_{CR}, F_{lo}, F_{hi}: SHADE memory and F bounds
- $p_{\max}$, $p_{\min}$, k_r^0, k_r^1: pbest and rank-pressure schedules
- B, p_{eig}: eigen-basis (Algorithm 3) and crossover probability
- n_{pool}: restricted-tournament pool size

OUTPUT
- Updated X, A, (M_F, M_{CR}), (x^*, f^*), ARQ success statistics

ARQ branch update
01 Sort X by fitness to obtain the rank order ord
02 If the eigen-basis is stale then refresh it (Algorithm 3)
03 Set m parents to try (dimension-dependent agent fraction) and draw them at random from ord
04 For each drawn parent $t = 1, \ldots, m$ sample (F_t, CR_t) from a random slot of (M_F, M_{CR})
05 If CR-sorting is enabled then reassign the sampled CR values so the best-ranked parents receive the smallest ones
06 For each selected parent i do
07 Draw x_{pbest} uniformly from the top $\lceil p(t)N \rceil$ individuals, $p(t)$ following the $p_{\max} \to p_{\min}$ schedule
08 Pick r_1 by rank-biased selection with pressure $k_r(t)$, forbidding i
09 With probability 0.5 draw r_2 from the archive A; otherwise pick r_2 by rank-biased selection, forbidding i, r_1
10 Compute the jSO weight $K(F_t)$ and form the mutant $v \leftarrow x_i + K(F_t)(x_{pbest} - x_i) + F_t(x_{r_1} - x_{r_2})$
11 Repair v to Ω
12 With probability p_{eig} apply eigen-coordinate binomial crossover using B; otherwise apply classical binomial crossover with rate CR_t to obtain the trial u
13 Repair u to Ω and evaluate $f(u)$
14 If $f(u) < f(x_i)$ then
15 Archive x_i, set $x_i \leftarrow u$, and record the success (F_t, CR_t, gain)
16 Else
17 Draw a pool $Q \subset X$ of size n_{pool} (excluding i) and find the nearest $q^\star \in Q$ to u
18 If $f(u) < f(q^\star)$ then archive $q^\star$, set $q^\star \leftarrow u$, inherit its IDE parameters from i, and record the success
19 Endif
20 Update (x^*, f^*) if improved
21 Endfor
22 Update the memory slot of (M_F, M_{CR}) from the recorded successes
23 Return the updated X, A, (M_F, M_{CR}), (x^*, f^*)

For each selected parent, a mutant is formed by pulling it toward a rank-biased elite exemplar plus a rank-biased difference term (lines 7–10), then crossed over either in the eigen-basis or in the raw coordinates (line 11). The trial first competes with its own parent and, failing that, with its nearest neighbor in a small random pool (lines 13–19), with all successes feeding the SHADE memory update (line 22).

Algorithm 5 Levy-flight quarantine in SPARQ

INPUT
- X, N, D, Ω: population, size, dimension, domain
- α_q, ρ_q, β_L, σ_L: IQR coefficient, repair fraction, Levy exponent, step scale

OUTPUT
- Updated X, A, (x^*, f^*)

Levy-flight quarantine
01 Compute Q_1, Q_3 and $IQR = Q_3 - Q_1$ of the fitness values
02 Set the threshold $\theta \leftarrow Q_3 + \alpha_q \cdot IQR$
03 Compute the centroid c of the best half of X by fitness
04 Set the outlier set $O \leftarrow \{i : f(x_i) \geq \theta\}$; if $O = \varnothing$ then return
05 Draw a random subset $O_\rho \subset O$ with $|O_\rho| = \lfloor \rho_q |O| \rfloor$
06 For each $i \in O_\rho$ do
07 For each coordinate j draw a Levy step η_j via the Mantegna algorithm with exponent β_L
08 Set $x_i' \leftarrow c + \sigma_L \cdot (\mathbf{u} - \ell) \odot \boldsymbol{\eta}$
09 Repair x_i' to Ω and evaluate $f(x_i')$
10 If $f(x_i') < f(x_i)$ then archive x_i, set $x_i \leftarrow x_i'$, and re-seed its IDE parameters
11 Update (x^*, f^*) if improved
12 Endfor
13 Return the updated X, A, (x^*, f^*)

Individuals whose fitness exceeds a robust outlier threshold (line 2) are relocated toward the population's best-half centroid using a heavy-tailed Lévy step rather than an ordinary Gaussian one (lines 7–8), giving them an occasional large jump to a genuinely different region. A relocation is only kept if it actually improves on the original individual (line 9).

Algorithm 6 Stagnation-gated elite polish in SPARQ

INPUT
- x^*, f^*, X, N, D, Ω: incumbent, population, size, dimension, domain
- B, $\mathbf{s}$: eigen-basis and per-axis scales (Algorithm 3)
- ρ_p, B_p: burst-size fraction and evaluation-budget cap
- σ_p, σ_c, b_{sw}, ρ_{sw}, σ_e: eigen-step, coordinate-step, Solis-Wets bias/step, echo scale

OUTPUT
- Updated x^*, f^*, X, adapted step sizes and cooldown state

Stagnation-gated elite polish
01 Set $f_{\text{before}} \leftarrow f^*$ and compute the evolutionary gain-per-evaluation since the last activation
02 Set the burst size $k \leftarrow \max(6, \text{round}(\rho_p N))$, enlarged late in the run to cover D coordinates
03 Set fails $\leftarrow 0$, wins $\leftarrow 0$
04 For $t = 1, \ldots, k$ do
05 If the evaluation budget or the cap B_p is exceeded then break
06 Draw a probe mode $m \sim U(0, 1)$
07 If $m < 0.30$ then probe a single round-robin coordinate (opposition or adaptive Gaussian step)
08 Else if $m < 0.65$ and B is available then draw an anisotropic Gaussian step in the eigen-basis, scaled by $\mathbf{s}$
09 Else if $m < 0.80$ and the echo buffer is non-empty then draw a random linear combination of the buffered directions (trajectory echo)
10 Else propose $x^* + b_{sw} + \delta$, and if it fails, the reflection $x^* - b_{sw} - \delta$ (Solis-Wets)
11 Repair the candidate to Ω and evaluate its fitness
12 If the candidate improves f^* then
13 Update (x^*, f^*), grow the step size of the active probe mode, replace the population's worst individual, reset fails, increment wins
14 Else shrink the step size of the active probe mode and increment fails
15 If fails ≥ 10 then break
16 Endfor
17 If wins $= 0$ then double the cooldown backoff; otherwise reset it
18 If the polish's gain-per-evaluation is below the evolutionary loop's then extend the cooldown
19 If the relative gain over the burst is negligible for several consecutive activations then extend the cooldown further
20 Return the updated x^*, f^*, X

While the incumbent best is stagnating, a short burst of local probes is applied directly to it, drawn from four alternative modes, coordinate-wise, eigen-aligned, trajectory-echo, and Solis-Wets (lines 6–10), and any improvement is written back into the population's worst slot (line 12). Three independent safeguards (lines 16–18) shut the mechanism down whenever it proves less productive than ordinary evolution.

Algorithm 7 On-demand OBL basin escape in SPARQ

INPUT
- X, x^*, N, D, Ω: population, incumbent, size, dimension, domain
- τ_s, ε_v, ρ_b, κ_b, s: stagnation trigger, spread-collapse ratio, escape fraction, cooldown, stagnation counter

OUTPUT
- Updated X, (x^*, f^*), s, cooldown state

OBL basin escape
01 If the cooldown counter is positive then decrement it and return
02 If not ($s \geq \tau_s$ and the normalized population spread $< \varepsilon_v$) then return
03 Sort the population by fitness
04 Set $count \leftarrow \max(1, \lfloor \rho_b N \rfloor)$
05 For each of the $count$ worst individuals x_i do
06 For each coordinate j compute the opposite point $\ell_j + u_j - x_{i,j}$
07 With probability 0.5 use the pure opposition; otherwise interpolate between the opposition and x^*
08 Repair the candidate to Ω and evaluate its fitness
09 If it improves x_i then archive x_i, replace it, and re-seed its IDE parameters
10 Update (x^*, f^*) if improved
11 Endfor
12 If any replacement was applied then reset the cooldown to κ_b and set $s \leftarrow 0$
13 Return the updated X, (x^*, f^*), s

This only fires when the search has stopped improving and the population has visibly collapsed onto one region (line 2). A fraction of the worst individuals are then reflected to the opposite side of the search domain (lines 6–7), and the stagnation counter is reset if this yields any actual improvement (line 10).

Algorithm 8 Hard-stagnation rejuvenation in SPARQ

INPUT
- X, x^*, N, D, Ω: population, incumbent, size, dimension, domain
- τ_s, m_r, κ_r, C_r, s: stagnation trigger, rejuvenation factor, keep-fraction, cooldown, stagnation counter

OUTPUT
- Updated X, (x^*, f^*), cooldown and escalation state

Hard-stagnation rejuvenation
01 If the cooldown counter is positive then decrement it and return
02 Compute the relative fitness spread f_{rel} across the population
03 Set collapsed $\leftarrow$ ($f_{\text{rel}} < \varepsilon_1$) or (spread $< \varepsilon_2$)
04 Set hard_stag $\leftarrow$ ($s \geq m_r \tau_s$) and (progress within its cutoff) and (spread bounded)
05 If collapsed and not hard_stag then update the weak-reseed streak; escalate to hard_stag once the streak exceeds its limit
06 Else reset the weak-reseed streak
07 Set survival $\leftarrow$ collapsed and not hard_stag; if neither holds then return
08 If hard_stag then update the strong-reseed failure streak; escalate to a near-full reset once it exceeds its limit
09 Sort the population by fitness
10 Set the elite fraction keep to preserve: a tiny sliver under a near-full reset, κ_r under hard_stag, or nearly all of N under survival
11 For each individual ranked below keep do
12 Draw a uniformly random point in Ω; archive the old individual and replace it; re-seed its IDE parameters
13 Update (x^*, f^*) if improved
14 Endfor
15 If hard_stag then reset the stagnation counter s and set a long cooldown; otherwise set a short cooldown
16 Return the updated X, (x^*, f^*)

A loose "survival" condition keeps re-seeding a small sliver of the population indefinitely to avoid the appearance of full convergence, while a much stricter "hard-stagnation" condition, reached only after prolonged non-improvement, re-seeds the large majority of the population (lines 2–10). Two escalation counters (lines 5, 8) force a stronger response whenever repeated firings of either condition keep failing to produce real gains.

Algorithm 9 NLPSR population-size reduction in SPARQ

INPUT
- X, N, N_{init}, $N_{\min}$, α_{NL}: population, current size, size bounds, shrink-shape exponent
- p: current progress ratio

OUTPUT
- Updated X and N

NLPSR population-size reduction
01 Compute the shrink-shape exponent $q \leftarrow 1 - (1 - \alpha_{\text{NL}})p$
02 Compute the target size $N_{\text{target}} \leftarrow \text{round}(N_{\text{init}} + (N_{\min} - N_{\text{init}})p^q)$, clamped to $[N_{\min}, N_{\text{init}}]$
03 If $N_{\text{target}} \geq N$ then return unchanged
04 Sort the population ascending by fitness
05 Keep the best N_{target} individuals (and their IDE memories); discard the rest
06 Invalidate the eigen-basis and set $N \leftarrow N_{\text{target}}$
07 Return the updated X, N

The overall control flow of SPARQ, summarizing the sequence of decisions executed within each iteration of the main loop, is illustrated in Figure 1. The diagram highlights the probabilistic selection between the ARQ and IDE branches, the mandatory Lévy-flight quarantine attached to the ARQ branch, and the three graduated stagnation-driven mechanisms, namely the stagnation-gated elite polish, the opposition-based basin escape, and the hard-stagnation rejuvenation, together with their respective activation gates.

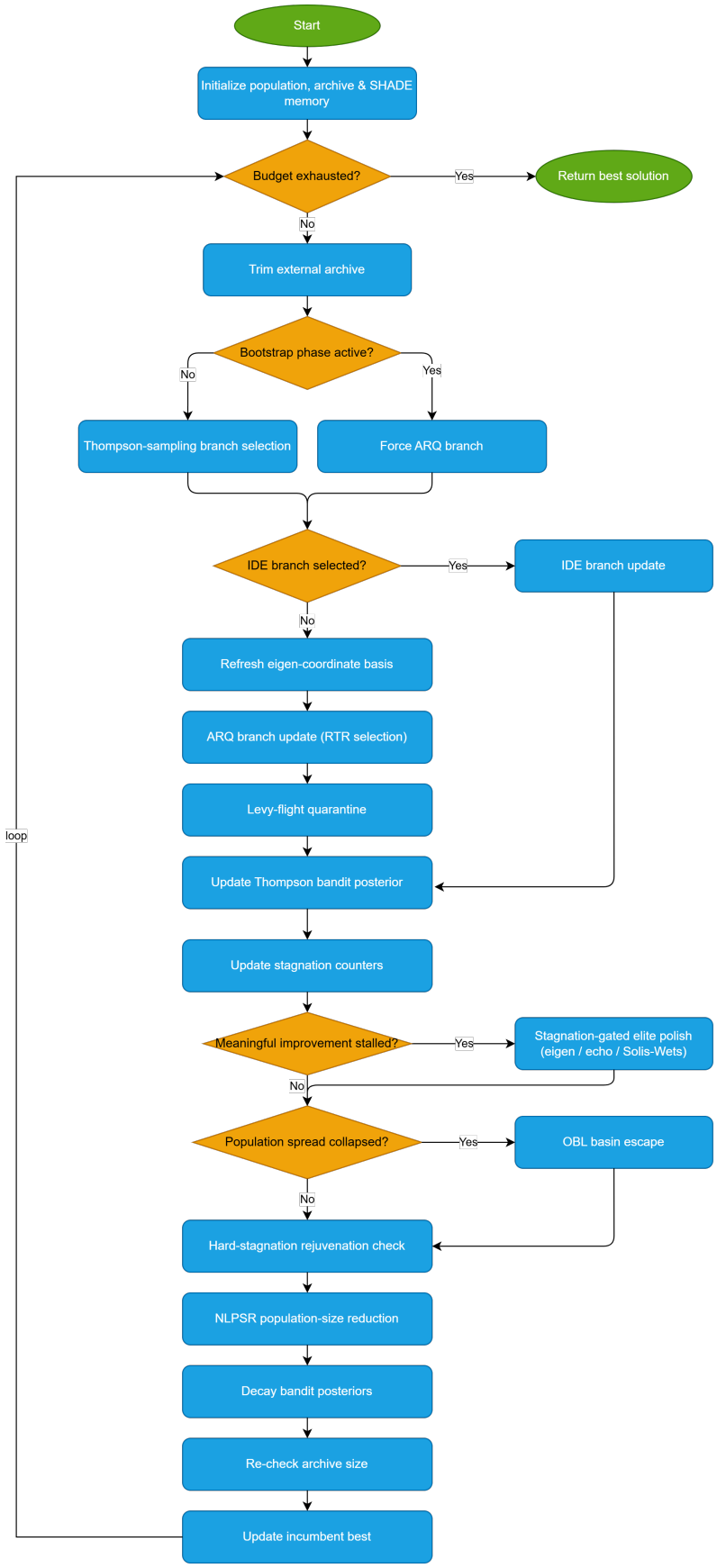


Figure 1. Overall control flow of the SPARQ algorithm, illustrating branch selection, the ARQ-attached quarantine mechanism, and the graduated stagnation-driven activation gates

The algorithm Figure 1 operates within a loop governed exclusively by the evaluation budget ($e < T_{eval}$). At the start of every pass through this loop, the external archive is first trimmed to size $\lfloor \alpha_A \cdot N \rfloor$, preventing it from growing unchecked as iterations accumulate.

The first substantive decision the system makes concerns which of the two search branches, ARQ or IDE, will be executed. As long as the bootstrap counter remains positive, the ARQ branch is enforced outright, with no genuine choice involved, guaranteeing a brief warm-up period before the bandit's statistics become reliable. Once this period elapses, branch selection passes to a Thompson-sampling bandit, which draws a random sample from each branch's posterior distribution and favors whichever branch returns the larger value. This selection is not a hard on/off switch but a probabilistic one: a branch that performs consistently worse will be chosen less often, yet it is never permanently excluded, since both posteriors decay toward the uninformative prior at every iteration, allowing a branch that was previously effective to be revisited later.

Depending on the outcome of this choice, execution diverges considerably. If the IDE branch is selected, only its own update is executed, with no accompanying mechanism. If the ARQ branch is selected, the eigen-basis is first checked for staleness and refreshed if necessary, the ARQ update itself is then executed, and the Lévy-flight quarantine correction is applied immediately afterward as a mandatory step. This quarantine mechanism is structurally attached exclusively to the ARQ branch and is never triggered when IDE executes, regardless of how often the latter is chosen. Both paths, however, converge on the same subsequent step: updating the bandit's posterior for whichever branch was just executed.

Two independent stagnation counters are then updated, each feeding a different downstream mechanism. The first records any absence of improvement whatsoever, however small, and feeds the basin-escape and rejuvenation mechanisms. The second records only the absence of meaningful, relative improvement, and feeds the elite-polish mechanism exclusively. This separation is deliberate: in large populations or at high dimensionality, some individual improves the incumbent by a negligible amount in nearly every generation, so a shared counter would almost never allow the polish phase to activate.

Three activation gates follow, always evaluated in the same order within a given iteration, and not mutually exclusive. The first concerns the elite polish, which is triggered once the second counter exceeds its stagnation threshold, once activated, it runs a short burst of local probes, alternating among single-coordinate probing, anisotropic eigen-aligned steps, replay from the trajectory-echo memory, and the classical Solis-Wets adaptive search, all protected by three independent safeguards: a hard evaluation-budget cap, an exponential cooldown whenever an entire burst fails to yield any improvement, and a gain-per-evaluation comparison against the ordinary evolutionary loop, which forces the polish to stand down whenever the core DE search proves the better use of the same evaluations. The second gate concerns the opposition-based basin-escape mechanism, which fires only when prolonged non-improvement and a visible collapse in population spread occur simultaneously, and only once its own cooldown has elapsed, if it produces no improvement, the cooldown is renewed without resetting the stagnation counter. The third gate, hard-stagnation rejuvenation, is not a simple binary decision but a hierarchical one: it internally evaluates whether only the milder survival condition holds, re-seeding a small fraction of the population at random without materially disrupting progress, or whether the stricter hard-stagnation condition holds, re-seeding the large majority of the population while preserving only its elite members. It further maintains its own escalation counters, so that repeated failures of the mild reseeding force an earlier transition to the harder condition, and repeated failures of the harder condition escalate to a near-complete reset.

Two further mechanisms execute at every iteration with no gating condition at all: population-size reduction via NLPSR, and the decay of the bandit's posteriors toward the uniform prior. NLPSR is always invoked, but effectively does nothing whenever the current target size is not smaller than the current population, so it remains continuously

active without always being consequential. The iteration closes with a second trimming of the archive, since the population size may have changed, and with an update to the current incumbent best, before the loop returns to its head until the evaluation budget is exhausted and the final solution is returned.

The overall picture that emerges is one of two permanently available search branches, whose relative usage is regulated probabilistically rather than through hard activation or deactivation, surrounded by an increasingly rare and increasingly disruptive layer of rescue mechanisms, each gated by strictly graduated stagnation conditions and each equipped with its own safeguards that switch it off automatically once it proves ineffective.

4. Experimental results

4.1. Experimental Configuration

All methods evaluated in this study, the proposed SPARQ algorithm together with the thirteen competing optimizers, were implemented in ANSI C++ within the open-source OptimSolution [31] framework, developed by Vasileios Charilogis and publicly available at https://github.com/charilog/optimsolution (accessed 18 July 2026). All experiments were compiled and executed under Debian 12.12 with GCC 13.4, on a workstation equipped with an AMD Ryzen 9 9950X3D processor (16 cores, 32 threads) and 64 GB of DDR4 memory. To account for the stochastic nature of the algorithms, each benchmark problem was solved over 30 independent runs, every run initialized with a distinct random seed. Across all runs and all methods, the evaluation budget was fixed at 150,000 function evaluations, consistent with the termination criterion adopted by the CEC 2011 real-world optimization benchmark [32] and allowing direct comparison with prior work. The best and mean objective values obtained at termination, averaged over the 30 independent runs per problem, are reported as the primary performance measures throughout the results section.

The success rate ("Rate%") reported in Table 3 onward is defined, for each problem-method combination, as the percentage of the 30 independent runs whose final best value falls within an adaptive tolerance of the best value obtained across that same batch of runs ("best-of-runs"): a run is counted as successful if $|f - f_{\text{best-of-runs}}| \leq \max\left(10^{-12}, 10^{-9}|f_{\text{best-of-runs}}|\right)$. This criterion measures within-method, within-problem consistency across repeated runs rather than proximity to the problem's true global optimum, which is not always known in closed form for the real-world engineering benchmarks, on problems where the best-of-runs value coincides with the known optimum, a high Rate% also reflects repeated convergence to that optimum. Feasibility is enforced at every evaluation by the repair and clamping procedures described in Section 2 onward, so all reported solutions already satisfy the problem's box constraints and no separate feasibility rate is needed.

Table 1 summarizes the SPARQ parameter settings actually used in these experiments, expressed in the notation introduced in Algorithm 1 and Sections 3.1–3.1, while Table 2 lists the internal constants that remain fixed at compile time and are therefore not exposed through the configuration interface.

Table 1. SPARQ parameter settings used in the experiments, following the notation introduced in Algorithm 1 and Sections 3.1–3.1

Symbol	Value	Description
General configuration		
N_{fe}	150,000	Evaluation budget per run
Runs	30	Independent repeated runs
Initialization	uniform	Population initialization distribution
Population / NLPSR (Section 3.1)		
N_{init}	100	Initial population size
N_{min}	4	Minimum population size
α_{NL}	0.5	NLPSR shrink-shape exponent

Table 1. SPARQ parameter settings used in the experiments, following the notation introduced in Algorithm 1 and Sections 3.1–3.1

Symbol	Value	Description
SHADE memory for F/CR (Section 3.1)		
H	6	SHADE circular memory size
M_F^H	0.9	Terminal memory slot for F
M_{CR}^H	0.9	Terminal memory slot for CR
F_{lo}	0.05	Lower bound for F sampling
F_{hi}	1.40	Upper bound for F sampling
jSO schedule / rank-based pressure (Section 3.1)		
p_{max}	0.25	Maximum p_{best} fraction
p_{min}	0.10	Minimum p_{best} fraction
$k_{r,init}$‡	2.0	Initial rank-selection pressure
$k_{r,final}$‡	3.0	Final rank-selection pressure
Eigen-coordinate crossover (Section 3.1, Algorithm 3)		
p_{eig}	0.40	Eigen-crossover probability
T_{eig}	10	Basis refresh period (iterations)
ρ_{eig}	0.50	Top-fraction used for the basis
D_{min}	2	Minimum dimension for eigen-basis activation
RTR selection / archive (Algorithm 4)		
n_{pool}	14	RTR neighbor pool size
ρ_m	0.10	Dynamic pool fraction of N (not named in the article)
α_A	1.5	Archive size factor
Thompson-sampling bandit (Section 3.1)		
γ_b	0.97	Bandit posterior decay rate
n_{bs}	2	Forced ARQ bootstrap iterations
Lévy-flight quarantine (Section 3.1, Algorithm 5)		
α_q	1.0	IQR outlier threshold coefficient
ρ_q	0.08	Fraction of outliers repaired
β_L	1.5	Lévy distribution exponent
σ_L	0.10	Lévy quarantine step scale
OBL basin escape (Section 3.1, Algorithm 7)		
τ_s	30	Stagnation threshold
ε_v	10^{-3}	Population-spread collapse ratio
ρ_b	0.30	Fraction of individuals relocated
κ_b	80	Cooldown after escape
Agent fraction (Algorithm 4, line 3)		
–	0.5	Fraction of parents tried per generation (m in Alg. 4; no dedicated symbol)
D_m	100	Dimension above which this fraction applies fully (not named in the article)
Stagnation-gated elite polish (Section 3.1, Algorithm 6)		
τ_p	10	Iterations before a polish burst fires
ρ_p	0.10	Polish burst size (probes)
B_p	0.12	Maximum evaluation share for polish
π_p	0.80	Late-phase threshold (trigger tightens to 1)
–	0.85	Progress fraction after which the burst enlarges (not separately named; grouped under π_p in Algorithm 1)
Hard-stagnation rejuvenation (Section 3.1, Algorithm 7)		
m_r	4	Hard-stagnation threshold multiplier
κ_r	0.25	Elite fraction preserved
C_r	150	Cooldown after rejuvenation

Table 1. SPARQ parameter settings used in the experiments, following the notation introduced in Algorithm 1 and Sections 3.1–3.1

Symbol	Value	Description
κ_{ws}	8	Failed weak-reseed streak limit (not named in the article)
κ_{hs}	2	Failed hard-reseed streak limit (not named in the article)
π_{off}	0.90	Progress fraction beyond which disabled (not named in the article)
CR-sorting (Section 3.3)		
–	enabled	Rank-based CR reassignment

Table 2. Internal constants of SPARQ fixed at compile time

Symbol	Value	Description
σ_p	0.02	Eigen-aligned probe step (Algorithm 6)
σ_c	0.10	Single-coordinate probe step (Algorithm 6)
ρ_{sw}	0.02	Solis–Wets probe step (Algorithm 6)
σ_e	1.0	Trajectory-echo scale (Algorithm 6)
σ_{lo}, σ_{hi}	10^{-9} / 0.25	Min./max. clamp for adaptive probe steps (not named in the article)
ε_{bo}	10^{-3}	Backoff threshold for low relative gain (not named in the article)
L_{echo}	8	Trajectory-echo buffer size (described in Section 3.1 as "eight entries", no symbol)
ε_{s_p}	10^{-6}	Meaningful-gain threshold for the s_p counter (not named in the article)

4.2. Comparative Results on Standard-Dimensionality Benchmark Problems

This subsection presents the outcome of the comparative evaluation on the standard-dimensionality portion of the benchmark suite. SPARQ is compared against nine state-of-the-art optimizers: ARQ2 [30], EA4Eig [24], jSO [18], LM-CMA-ES [9], mLSHADE-RL [33], NL-SHADE-LBC [34], SFCDE [35], TridentDE [36], and UDE3 [37]. The comparison covers forty-five benchmark problems. These include classical synthetic test functions alongside real-world engineering formulations, such as antenna array design, hydrothermal scheduling, economic load dispatch, and several chemical-engineering problems. Problem dimensionality in this subsection ranges from as few as five variables to several hundred. For each problem, every method was executed independently over a fixed number of repeated runs under an identical evaluation budget. Three complementary tables report the results. Table 3 presents detailed statistics for each method and problem, namely the best value found, the mean value, the success rate, the standard deviation, and the execution time. Table 4 reports the ranking of all ten methods on each problem according to the best value achieved. Table 5 reports the corresponding ranking according to the mean value, offering a stricter measure of reliability across repeated runs. Together, these three tables allow the performance of SPARQ to be assessed both in absolute terms and relative to its competitors, and both in terms of peak performance and average-case consistency.

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
alpine1 [d=100]	Best	6.6000e-11	5.3800e-09	1.7000e-11	0	1.2200e-10	0.00410456	1.6690e-08	0	0.000371752	1.8615e-07
	Mean	0.000176474	2.4673e-07	1.7126e-09	0	1.0039e-08	0.0223941	0.00086093	0	0.0033323	1.9353e-06
	Rate%	3%	3%	3%	100%	3%	3%	3%	100%	3%	3%
	SD	0.000615062	6.8327e-07	4.0570e-09	0	2.2429e-08	0.0268403	0.0047049	0	0.00380758	3.4386e-06
	Time(s)	0.319	6.438	0.153	0.1	0.174	0.165	0.338	0.164	0.834	0.677
antennaarray [d=12] [38]	Best	0.00680964	0.00680964	0.00680964	0.233659	0.00680964	0.00680964	0.00680964	0.00680964	0.00680964	0.00680964
	Mean	0.00681202	0.00680964	0.00680964	0.356811	0.00680964	0.00680964	0.0155814	0.00680964	0.00680975	0.00680964
	Rate%	67%	100%	100%	3%	100%	7%	93%	83%	13%	100%
	SD	7.4675e-06	1.7644e-18	1.7644e-18	0.0550951	1.7644e-18	2.2063e-08	0.0336259	1.7978e-10	4.1947e-07	1.7644e-18
	Time(s)	6.472	3.292	2.445	0.91	2.613	1.227	8.19	2.002	30.615	17.592
antennaula [d=10]	Best	0.156149	0.156149	0.156149	0.156308	0.156149	0.156149	0.156149	0.156149	0.156149	0.156149
	Mean	0.156149	0.156149	0.156149	0.156822	0.156149	0.156149	0.156149	0.156149	0.156149	0.156149
	Rate%	100%	100%	100%	3%	100%	93%	100%	100%	13%	100%
	SD	2.8230e-17	2.8230e-17	2.8230e-17	0.00037492	2.8230e-17	1.2181e-10	2.8230e-17	2.8230e-17	1.3557e-09	2.8230e-17
	Time(s)	12.108	6.358	4.825	1.626	4.64	2.424	16.327	3.991	56.682	32.536
bucherastrigin [d=50]	Best	3.03752	10.6264	7.9597	0	25.8689	16.9143	5.45194	0	42.7923	37.8084
	Mean	21.4995	18.457	25.1062	0	39.0687	71.4967	24.0398	0	87.4512	50.6661
	Rate%	3%	3%	3%	100%	3%	3%	3%	100%	3%	3%
	SD	10.5954	4.53841	17.0473	0	5.32883	32.3878	13.3618	0	31.335	6.69681
	Time(s)	0.472	2.25	0.202	0.097	0.218	0.17	0.547	0.241	1.66	1.182
cosinemixture [d=50] [38]	Best	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
	Mean	-5	-5	-5	-5	-4.99015	-4.95074	-4.78325	-5	-4.94089	-5
	Rate%	100%	100%	100%	100%	93%	70%	27%	100%	73%	100%
	SD	0	0	0	0	0.0374941	0.0807896	0.215154	0	0.113813	0
	Time(s)	0.242	0.563	0.155	0.149	0.164	0.167	0.208	0.19	0.482	0.468
ded1 [d=120] [38]	Best	130643	130643	130643	130774	130643	130855	130746	130643	130841	130650
	Mean	130645	130644	130643	130892	130644	130903	130788	130644	130905	130659
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	SD	1.95641	0.709733	0.163697	65.8632	0.895705	29.644	20.0706	0.719647	38.2176	7.13931
	Time(s)	0.382	0.236	0.205	0.13	0.184	0.164	0.399	0.165	0.944	0.778
ded2 **[d=216] [38]**	**Best**	165137	165128	164964	166873	165290	167727	166356	165202	165953	165108
	Mean	165656	165599	165626	168467	165639	168779	166961	165672	166519	165791
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	398.646	215.743	245.073	1248.51	237.79	428.514	316.334	265.268	288.062	248.323
	Time(s)	0.739	0.427	0.306	0.283	0.268	0.185	1.011	0.245	2.357	1.627
eld1 **[d=6] [38]**	**Best**	2967.25	2967.25	2967.25	2967.25	2967.25	2967.25	2967.25	2967.25	2967.25	2967.25
	Mean	2967.52	2967.54	2967.58	3190.82	2975.72	2967.6	2969.06	2969.13	2978.99	2974.84
	Rate%	57%	53%	47%	7%	17%	43%	57%	53%	3%	30%
	SD	0.310574	0.312675	0.312675	289.445	5.87434	0.310574	6.67985	4.26807	3.10799	6.18104
	Time(s)	0.113	0.145	0.164	0.177	0.158	0.17	0.111	0.164	0.209	0.283
eld2 **[d=13] [38]**	**Best**	17864.4	17864.7	17924.4	17936.7	17866.9	17924.4	17874.9	17874.5	17867.4	17879.7
	Mean	17881.5	17895.1	17954.8	18072	17881.4	18005.1	17917.2	17923.1	17938.9	17885.4
	Rate%	3%	7%	13%	3%	10%	3%	3%	3%	3%	10%
	SD	21.6346	31.1588	17.4242	88.4322	8.64524	58.2737	22.7828	32.4557	50.907	10.9639
	Time(s)	0.137	0.134	0.158	0.173	0.16	0.17	0.126	0.16	0.253	0.332
eld3 **[d=15] [38]**	**Best**	32367.6	32375.3	32458.9	32760.6	32367.6	32597.3	32401.5	32375.3	32367.6	32367.6
	Mean	32399.4	32425.9	32554	33110	32405.4	32798.9	32488.3	32440.5	32494	32434.2
	Rate%	7%	3%	3%	3%	53%	3%	3%	7%	13%	33%
	SD	29.7244	40.967	40.8494	208.751	74.2414	69.4704	53.4577	47.9137	100.649	73.6464
	Time(s)	0.151	0.151	0.155	0.171	0.16	0.17	0.145	0.156	0.286	0.355
eld4 **[d=40] [38]**	**Best**	121064	121096	121152	123426	121064	122082	121647	121096	121075	121075
	Mean	121188	121335	121390	125460	121181	123934	122353	121399	121403	121214
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	106.955	174.933	194.4	965.109	85.3427	816.891	376.605	202.602	191.838	116.902
	Time(s)	0.226	0.503	0.157	0.154	0.168	0.166	0.239	0.192	0.576	0.504
eld5 **[d=140] [38]**	**Best**	508614	508614	508614	510807	508614	508877	508715	508614	508659	508618

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	Mean	508619	508614	508614	511210	508616	509014	508786	508615	508702	508635
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	9.98242	0.106988	0.0852147	170.472	3.64487	79.002	31.9054	0.724255	26.958	15.0339
	Time(s)	0.416	0.395	0.375	0.136	0.18	0.168	0.4	0.162	0.861	0.724
fmsynth [d=6] [38]	**Best**	0	0	0	0.274042	0.116158	0.000131621	9.6570e-06	0	0.116158	6.6499e-08
	Mean	0.115033	0.098221	0.0972373	0.331597	0.121941	0.10375	0.108498	0.105976	0.126956	0.115608
	Rate%	3%	17%	10%	3%	83%	3%	3%	10%	73%	3%
	SD	0.0228626	0.0453469	0.0442939	0.0236314	0.0171161	0.0337617	0.0329245	0.0363381	0.023476	0.0245231
	Time(s)	0.47	0.28	0.226	0.151	0.219	0.179	0.657	0.208	1.859	1.356
gallagher101 [d=16] [38]	**Best**	0	0	0	0.691857	0.691857	0.929997	0	0.929997	0	0
	Mean	1.81765	1.88988	2.34955	9.20322	1.88241	4.38908	3.25587	3.11339	2.9818	2.21648
	Rate%	13%	10%	3%	7%	3%	23%	7%	37%	13%	3%
	SD	1.59835	1.6648	1.72389	10.2299	1.49672	4.17408	2.95444	2.73578	3.50141	1.61107
	Time(s)	1.789	0.857	0.63	0.256	0.633	0.324	2.175	0.565	7.359	4.419
gallagher21 [d=16]	**Best**	0	0	0	0	0	0	0	0	0	0
	Mean	4.99458	1.72255	6.80626	10.9842	3.85307	8.74813	5.86152	4.7033	5.23155	5.2396
	Rate%	50%	80%	53%	17%	43%	33%	53%	53%	30%	37%
	SD	6.63609	4.13367	7.40059	5.64072	6.16112	6.74433	8.16359	6.14221	5.88774	6.28461
	Time(s)	0.474	0.24	0.196	0.115	0.214	0.172	0.539	0.206	1.668	1.128
griewankrosenbrock [d=50]	**Best**	0.975471	2.99495	1.64009	1.99363	16.6786	0.545139	2.9697	0.756589	7.68858	25.3427
	Mean	2.10878	5.82533	2.22636	4.73453	24.7386	0.773744	3.7164	1.60007	10.9174	27.4377
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	0.715515	1.72956	0.49828	2.25592	2.28942	0.144207	0.281837	0.525703	1.50984	1.01623
	Time(s)	0.265	2.152	0.15	0.145	0.165	0.164	0.24	0.239	0.671	0.602
heatexchanger [d=22]	**Best**	137715	137682	137682	137682	137682	137715	137715	137715	137682	137682
	Mean	137715	137687	137713	139785	137682	137715	137715	137715	137682	137682
	Rate%	100%	83%	7%	33%	100%	100%	100%	100%	100%	100%
	SD	0	12.6233	8.4491	2930.89	8.8804e-11	0	0	0	8.8804e-11	8.8804e-11

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	Time(s)	0.161	0.305	0.131	0.172	0.163	0.164	0.145	0.176	0.265	0.355
hydrothermal [d=120] [38]	**Best**	141656	141670	141662	231975	141656	145194	141841	141656	141795	141672
	Mean	141660	141694	141675	351889	141659	151361	142079	141669	141876	141704
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	6.43564	11.5562	10.117	67903	2.94957	3944.21	299.956	9.8093	85.9947	18.8546
	Time(s)	0.365	0.411	0.444	0.126	0.175	0.169	0.378	0.172	0.889	0.717
katsuura [d=50]	**Best**	0.019147	0	0	0	0	0.00735398	0.539615	0	0	0
	Mean	0.0712054	6.3333e-13	0	0	0	0.471516	0.660281	0	0	0
	Rate%	3%	97%	100%	100%	100%	3%	3%	100%	100%	100%
	SD	0.0768892	3.2851e-12	0	0	0	1.66103	0.0674435	0	0	0
	Time(s)	2.934	2.056	1.153	0.489	0.891	0.469	3.178	0.836	10.835	6.513
levy [d=100]	**Best**	0.805754	0.0895283	0.358113	2.16204	0.626698	2.68592	2.97448	0.17911	1.70104	0.268585
	Mean	2.14332	0.382655	1.2195	8.27654	1.28239	3.67756	5.40488	1.26403	3.22232	1.11193
	Rate%	7%	17%	3%	3%	7%	3%	7%	3%	3%	3%
	SD	0.8117	0.244084	0.535975	2.04575	0.564979	0.817505	2.09253	0.598227	1.31638	0.457073
	Time(s)	0.397	6.053	0.161	0.112	0.182	0.161	0.408	0.165	1.052	0.823
messenger [d=14]	**Best**	26.7175	26.7175	26.7175	26.862	26.7175	26.7175	26.7175	26.7175	26.7175	26.7175
	Mean	26.7175	26.7175	26.7175	27.2064	26.7175	26.7175	26.7175	26.7175	26.7175	26.7175
	Rate%	100%	100%	100%	3%	100%	100%	100%	100%	100%	100%
	SD	7.2269e-15	7.2269e-15	7.2269e-15	0.220782	7.2269e-15	4.0769e-13	7.2269e-15	7.2269e-15	7.2269e-15	7.2269e-15
	Time(s)	0.126	0.147	0.163	0.176	0.159	0.169	0.13	0.168	0.207	0.303
michalewicz [d=100]	**Best**	-96.0326	-77.4438	-93.8523	-79.5563	-95.7084	-99.2415	-79.5299	-98.6917	-65.359	-43.2703
	Mean	-93.0301	-69.5914	-91.3666	-72.7176	-93.5333	-99.1589	-77.7664	-97.0668	-59.528	-39.8999
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	3.12945	3.34186	1.04978	4.43642	1.13955	0.0714478	0.795036	0.964969	2.01878	1.40427
	Time(s)	0.783	18.223	0.301	0.174	0.29	0.186	0.815	0.227	2.615	1.863
polyphase [d=20] [38]	**Best**	0.0290567	0.0201421	0.00571762	0.284493	0.023238	0.108972	0.314647	0.000621833	0.0475955	0.0298618
	Mean	0.294367	0.128674	0.2011	0.807162	0.185771	0.342068	0.5693	0.101974	0.298147	0.254132

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	0.114409	0.100273	0.12277	0.293895	0.127557	0.0817433	0.101133	0.0639291	0.164167	0.168076
	Time(s)	0.313	0.269	0.182	0.161	0.189	0.165	0.473	0.189	0.946	0.832
potential [d=38]	**Best**	-131.443	-155.503	-160.182	-168.198	-149.112	-157.276	-110.769	-167.054	-127.985	-73.2198
	Mean	-116.772	-142.71	-149.977	361.377	-110.74	-153.238	-105.833	-160.986	-101.665	39.4225
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	6.09232	7.18836	4.25891	620.768	27.1126	2.07754	2.85937	3.71072	13.6216	84.9164
	Time(s)	0.754	0.459	0.824	0.173	0.232	0.18	0.637	0.197	2.032	1.383
powell [d=100]	**Best**	0.000306326	0.000406189	8.8867e-05	2.2919e-06	0.000653703	0.0703045	0.0208072	0	0.0745429	0.0121888
	Mean	0.00210306	0.00112813	0.000183153	1.1710e-05	0.00138046	0.307694	0.0387934	0	0.204422	0.0183201
	Rate%	3%	3%	3%	3%	3%	3%	3%	100%	3%	3%
	SD	0.00330406	0.000392613	6.6716e-05	5.3207e-06	0.000354841	0.120565	0.00945123	0	0.0953043	0.00387703
	Time(s)	0.273	12.152	0.153	0.096	0.166	0.165	0.282	0.158	0.589	0.554
rosenbrock [d=50]	**Best**	4.29661	17.7119	0.0552813	8.81167	12.7153	38.3447	19.14	14.951	36.2386	29.2749
	Mean	13.6938	21.4993	2.46021	12.4255	16.8552	40.5928	23.1997	24.7064	37.3863	31.35
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	6.40239	1.87024	2.04484	2.48772	2.23124	1.14897	2.82364	3.1607	0.608985	0.852551
	Time(s)	0.19	1.831	0.153	0.157	0.16	0.164	0.174	0.235	0.452	0.404
rosetta [d=22]	**Best**	3.36872	3.36872	3.72444	3.81703	3.36872	3.72444	3.36872	3.36872	3.36872	3.36872
	Mean	3.67701	3.71259	3.72444	5.13858	3.45172	3.72444	3.36872	3.36872	3.49915	3.48729
	Rate%	13%	3%	100%	3%	77%	80%	100%	100%	63%	67%
	SD	0.122991	0.0649465	1.3550e-15	0.77485	0.153028	4.3759e-08	9.0336e-16	9.0336e-16	0.174353	0.170558
	Time(s)	0.153	0.174	0.17	0.172	0.163	0.167	0.152	0.174	0.269	0.348
rotatedrosenbrock [d=50]	**Best**	0.00604507	17.1324	6.8860e-09	1.9000e-11	1.7355e-07	0.021825	0.000383499	0.000444079	0.00248885	0.037282
	Mean	18.4717	24.1876	3.76344	9.01549	19.4448	18.0046	42.9691	27.671	27.6186	43.6173
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	17.4377	10.0064	3.00886	5.33793	23.5759	27.9477	42.3238	15.0013	33.354	33.4042
	Time(s)	0.193	1.9	0.151	0.145	0.16	0.168	0.203	0.237	0.527	0.418

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
salomon [d=100]	Best	0.499873	0.799873	0.699873	0	0.899873	3.49987	0.499873	0	1.19987	0.699873
	Mean	1.16987	1.05321	0.959873	0	1.18321	4.58654	0.86654	0	1.67654	0.859874
	Rate%	3%	7%	3%	100%	7%	3%	3%	100%	3%	3%
	SD	0.313105	0.127937	0.132873	0	0.153316	0.644196	0.250975	0	0.286095	0.106996
	Time(s)	0.361	13.864	0.147	0.094	0.169	0.164	0.283	0.165	0.574	0.566
schwefel [d=24]	Best	0.000305462	0.000305462	0.000305462	3552.85	0.000305462	0.000305462	0.000305462	0.000305462	0.000305462	0.00207051
	Mean	0.000305462	0.000305462	129.625	4825.87	51.3236	43.4277	0.000305462	0.000305462	604.44	2970.62
	Rate%	53%	100%	10%	3%	63%	63%	77%	90%	3%	3%
	SD	1.2576e-12	0	192.272	681.49	74.1498	79.1968	8.6037e-13	6.1026e-13	346.058	780.031
	Time(s)	0.198	0.24	0.16	0.17	0.16	0.169	0.156	0.168	0.396	0.437
sinusoidal [d=200]	Best	-3.5	-3.5	-3.5	-3.5	-3.49999	-2.94397	-3.49938	-3.5	-3.34223	-3.49888
	Mean	-3.49932	-3.49999	-3.5	-1.01881	-3.49993	-2.24821	-3.46135	-3.5	-3.11639	-3.49466
	Rate%	3%	3%	7%	20%	3%	3%	3%	100%	3%	3%
	SD	0.00204778	6.7328e-06	4.3683e-06	1.33665	8.0365e-05	0.290855	0.148488	0	0.44314	0.00396205
	Time(s)	0.682	0.333	0.256	0.261	0.268	0.19	0.84	0.234	2.355	1.627
stirredtankreactor [d=20] [38]	Best	0.534576	0.534576	0.534576	0.578457	0.534576	0.534576	0.534576	0.534576	0.534576	0.534576
	Mean	0.534576	0.534576	0.534576	0.655901	0.534576	0.534576	0.534576	0.534576	0.534576	0.534576
	Rate%	100%	100%	100%	3%	100%	100%	100%	100%	100%	100%
	SD	2.2584e-16	2.2584e-16	2.2584e-16	0.0439165	2.2584e-16	2.2584e-16	2.2584e-16	2.2584e-16	2.2584e-16	2.2584e-16
	Time(s)	0.151	0.167	0.149	0.171	0.161	0.166	0.179	0.174	0.299	0.36
tandem [d=18]	Best	27.6131	27.6131	27.6131	27.9359	27.6131	27.6131	27.6131	27.6131	27.6131	27.6131
	Mean	27.6131	27.6131	27.6131	28.5521	27.6131	27.6136	27.6131	27.6131	27.6131	27.6131
	Rate%	100%	100%	100%	3%	100%	3%	3%	100%	100%	100%
	SD	7.2269e-15	7.2269e-15	7.2269e-15	0.384512	7.2269e-15	0.000849244	1.4502e-05	7.2269e-15	2.5431e-13	7.2269e-15
	Time(s)	0.133	0.16	0.159	0.172	0.161	0.168	0.129	0.17	0.248	0.321
tersoffb [d=30]	Best	-36.7227	-35.6542	-36.4017	-33.0671	-36.0879	-35.8773	-34.4524	-36.0073	-35.4998	-36.4459
	Mean	-34.8526	-34.5904	-34.7363	-27.6905	-33.99	-34.9479	-33.7635	-34.0183	-33.8424	-33.5724
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	SD	0.751157	0.855449	0.69871	3.85757	1.12758	0.391396	0.34209	1.37822	1.02464	1.22774
	Time(s)	2.013	1.387	0.873	0.243	0.682	0.454	2.699	0.629	6.956	5.091
tersoffc [d=30]	**Best**	-36.2305	-37.1606	-37.6202	-34.5494	-36.0066	-35.8429	-35.2832	-35.7001	-37.1504	-35.375
	Mean	-35.1432	-35.0916	-35.1291	-28.9997	-34.4414	-35.351	-34.0384	-33.9619	-34.4759	-33.8948
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	0.547894	0.791315	0.748068	4.33677	0.959211	0.253253	0.407002	1.02789	1.06793	0.748882
	Time(s)	3.449	2.074	1.535	0.494	1.258	0.826	4.642	1.025	13.451	8.976
test2n [d=200]	**Best**	-7620.59	-7725.73	-7126.4	-6829.53	-7394.99	-7733.32	-7069.81	-7833.23	-6884.53	-7209.33
	Mean	-7150.57	-7533.34	-6937.91	-6641.04	-7240.39	-6914.4	-6914.21	-7833.23	-6642.31	-7016.36
	Rate%	3%	3%	3%	3%	3%	3%	3%	53%	3%	3%
	SD	263.653	99.0168	111.766	104.566	107.332	315.232	86.5434	0.000515372	109.786	85.293
	Time(s)	0.511	0.361	0.227	0.191	0.192	0.17	0.473	0.175	1.012	0.901
test30n [d=200]	**Best**	4.4641e-08	1.2776e-07	1.3586e-08	0	3.1773e-06	6.35398	0.000197792	0	0.874805	0.0075017
	Mean	0.0426922	0.00797665	0.00872158	0.00686355	0.0521622	12.0216	0.401587	0.0131041	2.6814	0.107771
	Rate%	3%	3%	3%	57%	3%	3%	3%	37%	3%	3%
	SD	0.0831271	0.0222685	0.0108496	0.0100529	0.118902	2.39491	0.354119	0.0286558	1.2027	0.116349
	Time(s)	0.61	0.265	0.211	0.223	0.235	0.174	0.69	0.216	1.852	1.359
tnep [d=7] [38]	**Best**	21	21	21	23	21	21	21	21	21	21
	Mean	21	21	21	27.3333	21	21.5667	21.1	21	21.1667	21
	Rate%	100%	100%	100%	13%	100%	67%	90%	100%	83%	100%
	SD	0	0	0	3.16591	0	1.10433	0.305129	0	0.379049	0
	Time(s)	0.204	0.136	0.138	0.169	0.153	0.158	0.254	0.162	0.467	0.527
transmissionpricing [d=126] [38]	**Best**	4.53605	4.53605	4.53605	4.54397	4.53606	4.58931	4.53605	4.53607	4.54869	4.5466
	Mean	4.54089	4.53903	4.53606	4.71216	4.53784	4.83932	4.60599	4.55727	4.62265	4.6025
	Rate%	10%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	0.00994537	0.00455269	1.7330e-05	0.103526	0.00358582	0.560773	0.0499372	0.0214215	0.0432556	0.0453235
	Time(s)	3.649	2.127	1.538	1.014	1.536	0.773	5.272	1.001	17.902	10.47
trid [d=40]	**Best**	-11440	-11440	-11440	-11440	-11373.5	-8588.73	-9720.71	-11440	-10532	-11399.9

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	Mean	-11331.6	-11439.6	-11439.9	-11440	-11230.3	-3989.34	-5355.66	-11399.3	-4500.91	-10543.8
	Rate%	3%	3%	3%	100%	3%	3%	3%	3%	3%	3%
	SD	289.641	0.642353	0.145549	2.3668e-10	113.783	1964.33	2929.99	78.7711	3426.05	559.18
	Time(s)	0.167	0.955	0.155	0.157	0.159	0.167	0.291	0.233	0.347	0.366
vibratingplatform [d=5] [39]	**Best**	0.105406	0.105406	0.105406	0.174408	0.105406	0.105406	0.105406	0.105406	0.105406	0.105406
	Mean	0.105406	0.105406	0.105406	0.254461	0.105406	0.105407	0.105409	0.105406	0.10541	0.105406
	Rate%	3%	3%	3%	3%	3%	3%	3%	3%	3%	3%
	SD	3.7548e-07	4.3623e-07	1.2843e-07	0.0615324	3.5987e-07	3.4385e-06	5.3869e-06	7.9169e-07	6.1354e-06	3.1740e-07
	Time(s)	0.106	0.144	0.162	0.18	0.157	0.172	0.107	0.163	0.245	0.284
weatherirrigation [d=30] [40]	**Best**	2736.36	2736.36	2737.8	2743.85	2736.36	2751.62	2741.22	2736.36	2736.36	2736.36
	Mean	2745.94	2740.61	2744.38	2783.55	2751.4	2817.49	2773.37	2741.69	2763.76	2750.55
	Rate%	3%	20%	3%	3%	13%	3%	3%	20%	3%	3%
	SD	8.72179	4.04687	5.79116	25.4044	14.0046	29.1568	17.7335	5.56177	20.5874	10.5355
	Time(s)	0.459	0.336	0.191	0.094	0.214	0.174	0.537	0.212	1.616	1.1
weierstrass [d=70]	**Best**	0.0444884	0.00735326	0.0308165	0	0.196602	14.0741	1.66031	0	2.39297	0.166803
	Mean	3.11574	0.2273	0.944241	0	0.935228	17.1846	3.91315	0	4.75133	0.776516
	Rate%	3%	3%	3%	100%	3%	3%	3%	100%	3%	3%
	SD	2.21603	0.251044	0.769111	0	0.505221	1.82463	1.14477	0	1.41729	0.432902
	Time(s)	7.171	5.336	2.923	1.647	2.88	1.505	9.978	2.064	34.916	20.088
whitley [d=40]	**Best**	170.351	263.406	263.305	735.916	508.102	615.033	133.182	96.4938	152.941	575.902
	Mean	382.149	414.062	488.063	735.916	639.163	1098.28	517.969	396.182	593.577	656.457
	Rate%	3%	3%	3%	100%	3%	3%	3%	3%	3%	3%
	SD	146.202	90.2828	129.892	4.6252e-13	50.325	146.727	166.89	122.316	173.816	34.5759
	Time(s)	1.78	1.929	0.729	0.339	0.682	0.334	2.362	0.6	8.195	4.926
wirelesscoverage [d=6]	**Best**	0.946351	0.946351	0.946351	0.96	0.946351	0.946351	0.946351	0.946351	0.946351	0.946351
	Mean	0.946439	0.946361	0.946403	1.01894	0.946432	0.947578	0.946868	0.946351	0.946523	0.946432
	Rate%	33%	97%	50%	3%	73%	3%	7%	97%	43%	73%
	SD	0.00042572	5.5557e-05	0.000114485	0.0435639	0.000136866	0.00117242	0.00072299	1.0713e-06	0.000153368	0.000136866

Table 3. Detailed Performance Comparison of SPARQ Against Seven State-of-the-Art Optimizers on Standard and Real-World Benchmark Problems

PROBLEM	METRIC	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
	Time(s)	7.885	4.239	3.139	0.928	3.237	1.543	10.487	2.66	35.607	21.123

Table 3 reports Best, Mean, Rate%, SD, and execution time for all ten methods on each of the forty-five problems. Aggregated across the full set, SPARQ attains a success rate of 100% on 15 problems, or 33.3% of the total. Its average success rate is 46.6%, the highest figure among the ten methods. The next-best average success rates are 31.4% for mLSHADE-RL, 30.4% for EA4Eig, and 27.6% for UDE3. ARQ2, LM-CMA-ES, SFCDE, TridentDE, and NL-SHADE-RSP each average below 25%. The magnitude of this advantage is clearly visible on individual problems. On alpine1 (d=100), bucherastrigin (d=50), salomon (d=100), and weierstrass (d=70), SPARQ and LM-CMA-ES jointly reach a mean error of 0 at 100% success. The remaining eight methods are confined to 3 to 7% success. Their mean errors on bucherastrigin, for instance, range from 18.46 for EA4Eig to 87.45 for TridentDE. On powell (d=100) and sinusoidal (d=200), SPARQ alone reaches 100% success, with a mean error of 0 and -3.5 respectively. Every other method is held to 3 to 20% success on these two problems. LM-CMA-ES in particular sees its sinusoidal mean collapse to -1.02. On the real-world load-dispatch and engineering problems, namely eld2 through eld5, hydrothermal, tersoffb, tersoffc, and transmissionpricing, every method in the comparison records success rates no higher than 3 to 10%. SPARQ is no exception here. This pattern indicates that the difficulty is intrinsic to these landscapes rather than a shortcoming of any single method. In terms of computational cost, SPARQ's cumulative execution time across all forty-five problems is 22.04 seconds. This is the second-lowest total among the ten methods, behind only LM-CMA-ES at 13.47 seconds. ARQ2 requires 60.95 seconds, SFCDE 78.01 seconds, EA4Eig 102.45 seconds, UDE3 157.60 seconds, and TridentDE 254.43 seconds. TridentDE alone requires roughly eleven times SPARQ's total runtime for the same benchmark set.

Table 4. Best-Value Ranking of SPARQ and Nine Competing Metaheuristics Across the Full Benchmark Suite

PROBLEM	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
alpine1	3	6	3	3	3	10	7	3	9	8
antennaarray	5	5	5	10	5	5	5	5	5	5
antennaula	5	5	5	10	5	5	5	5	5	5
bucherastrigin	3	6	5	1.5	8	7	4	1.5	10	9
cosinemixture	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5
ded1	4	5	1	8	2	10	7	3	9	6
ded2	4	3	1	9	6	10	8	5	7	2
eld1	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5
eld2	1	2	8.5	10	3	8.5	6	5	4	7
eld3	2.5	5.5	8	10	2.5	9	7	5.5	2.5	2.5
eld4	2	5.5	7	10	1	9	8	5.5	4	3
eld5	5	3	1	10	2	9	8	4	7	6
fmsynth	2.5	2.5	2.5	10	8.5	7	6	2.5	8.5	5
gallagher101	3.5	3.5	3.5	7.5	7.5	9.5	3.5	9.5	3.5	3.5
gallagher21	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5	5.5
griewankrosenbrock	3	7	4	5	9	1	6	2	8	10
heatexchanger	8.5	3.5	3.5	3.5	3.5	8.5	8.5	8.5	3.5	3.5
hydrothermal	2	5	4	10	1	9	8	3	7	6
katsuura	9	4	4	4	4	8	10	4	4	4
levy	6	1	4	8	5	9	10	2	7	3
messenger	5	5	5	10	5	5	5	5	5	5
michalewicz	3	8	5	6	4	1	7	2	9	10
polyphase	5	3	2	9	4	8	10	1	7	6
potential	7	5	3	1	6	4	9	2	8	10
powell	4	5	3	2	6	9	8	1	10	7
rosenbrock	2	6	1	3	4	10	7	5	9	8
rosetta	4	4	8.5	10	4	8.5	4	4	4	4
rotatedrosenbrock	7	10	2	1	3	8	4	5	6	9
salomon	3	7	5.5	1.5	8	10	4	1.5	9	5.5
schwefel	4.5	4.5	4.5	10	4.5	4.5	4.5	4.5	4.5	9
sinusoidal	3	5	4	1.5	6	10	7	1.5	9	8
stirredtankreactor	5	5	5	10	5	5	5	5	5	5
tandem	4	4	4	10	4	9	8	4	4	4
tersoffb	1	7	3	10	4	6	9	5	8	2
tersoffc	4	2	1	10	5	6	9	7	3	8
test2n	4	3	7	10	5	2	8	1	9	6
test30n	4	5	3	1.5	6	10	7	1.5	9	8
tnep	5	5	5	10	5	5	5	5	5	5
transmissionpricing	1	4	2	7	5	10	3	6	9	8
trid	4	3	1.5	1.5	7	10	9	5	8	6
vibratingplatform	7.5	3.5	3.5	10	3.5	3.5	9	7.5	3.5	3.5
weatherirrigation	3.5	3.5	7	9	3.5	10	8	3.5	3.5	3.5
weierstrass	5	3	4	1.5	7	10	8	1.5	9	6
whitley	4	6	5	10	7	9	2	1	3	8
wirelesscoverage	5	5	5	10	5	5	5	5	5	5

Table 4 reports the ranking of each method according to its best-value result, with tied values assigned the average of the ranks they jointly occupy: for example, five methods tied for first place each receive rank 3, the mean of ranks 1 through 5, rather than all five being assigned rank 1. It covers the same ten methods and forty-five problems. Under this convention, SPARQ is the sole first-place method, with no ties, on 4 of the 45 problems (8.9%). jSO attains sole first place more often, on 5 of 45 (11.1%), the highest exclusive share

among the ten methods, and ARQ2 on 3 of 45 (6.7%). SPARQ's advantage instead lies in its average rank across the full set: its mean rank of 4.02 is the lowest of the ten methods, ahead of jSO at 4.13 and ARQ2 at 4.22, indicating that SPARQ participates in the many-way ties for first place, chiefly on separable problems where several methods jointly reach the global optimum, at least as consistently as its closest competitors, even though it is not the most frequent sole winner. At the other extreme, NL-SHADE-LBC and LM-CMA-ES average 7.32 and 6.92 respectively. They register the worst rank, 10th, on 11 problems (24.4%) and 20 problems (44.4%) respectively.

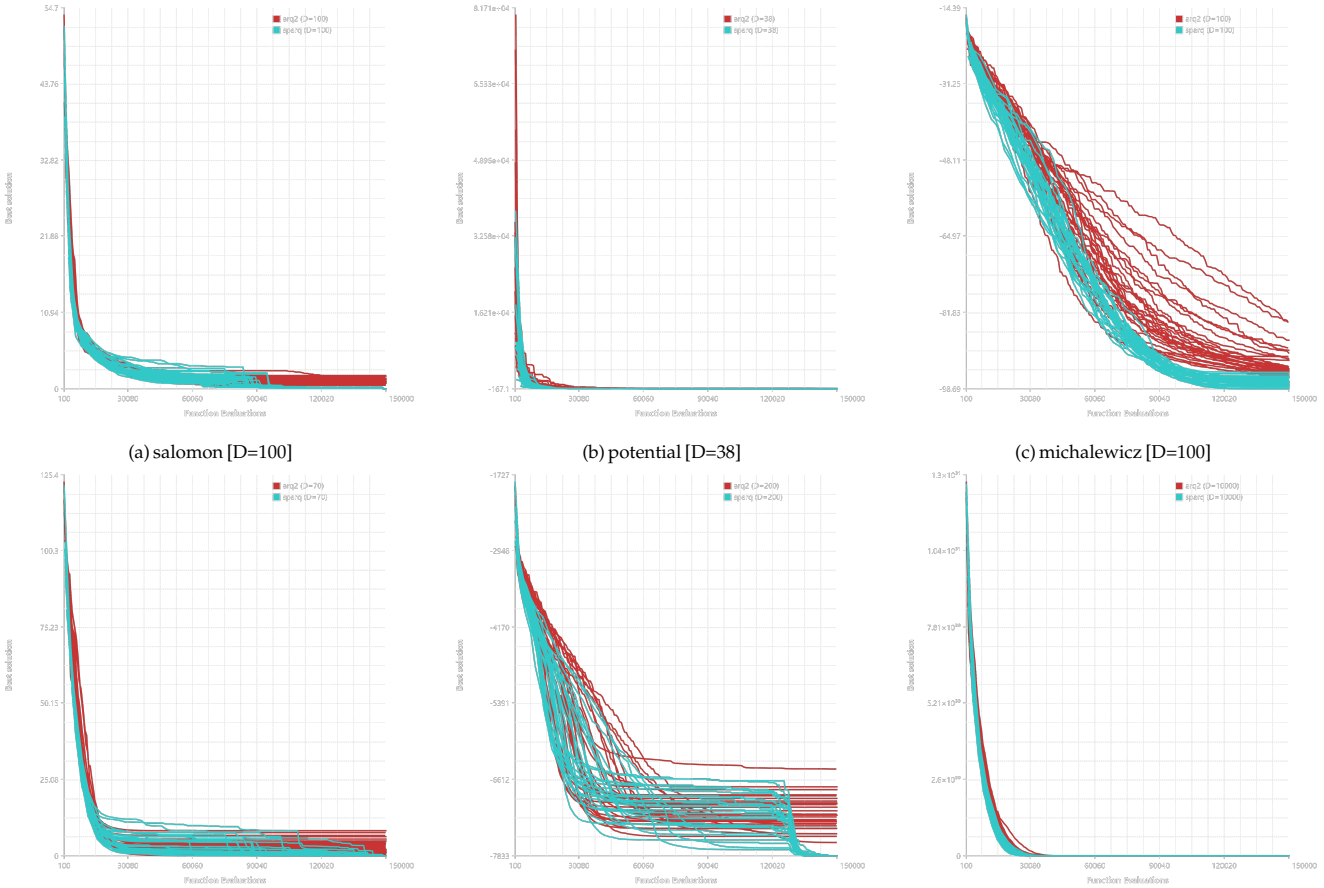
Table 5. Mean-Value Ranking of SPARQ and Nine Competing Metaheuristics Across the Full Benchmark Suite

PROBLEM	arq2	ea4eig	jso	lmcmaes	mlshaderl	nlshadelbc	sfcde	sparq	tridentde	ude3
alpine1	7	5	3	1.5	4	10	8	1.5	9	6
antennaarray	8	3	3	10	3	6	9	3	7	3
antennaula	4.5	4.5	4.5	10	4.5	4.5	4.5	4.5	9	4.5
bucherastrigin	4	3	6	1.5	7	9	5	1.5	10	8
cosinemixture	3.5	3.5	3.5	3.5	7	8	10	3.5	9	3.5
ded1	5	4	1	8	2	9	7	3	10	6
ded2	4	1	2	9	3	10	8	5	7	6
eld1	1	2	3	10	8	4	5	6	9	7
eld2	2	4	8	10	1	9	5	6	7	3
eld3	1	3	8	10	2	9	6	5	7	4
eld4	2	4	5	10	1	9	8	6	7	3
eld5	5	2	1	10	4	9	8	3	7	6
fmsynth	6	2	1	10	8	3	5	4	9	7
gallagher101	1	3	5	10	2	9	8	7	6	4
gallagher21	4	1	8	10	2	9	7	3	5	6
griewankrosenbrock	3	7	4	6	9	1	5	2	8	10
heatexchanger	7.5	4	5	10	2	7.5	7.5	7.5	2	2
hydrothermal	2	5	4	10	1	9	8	3	7	6
katsuura	8	4	4	4	4	9	10	4	4	4
levy	6	1	3	10	5	8	9	4	7	2
messenger	5	5	5	10	5	5	5	5	5	5
michalewicz	4	8	5	7	3	1	6	2	9	10
polyphase	6	2	4	10	3	8	9	1	7	5
potential	5	4	3	10	6	2	7	1	8	9
powell	6	4	3	2	5	10	8	1	9	7
rosenbrock	3	5	1	2	4	10	6	7	9	8
rosetta	6	7	8	10	3	9	1.5	1.5	5	4
rotatedrosenbrock	4	6	1	2	5	3	9	8	7	10
salomon	7	6	5	1.5	8	10	4	1.5	9	3
schwefel	2.5	2.5	7	10	6	5	2.5	2.5	8	9
sinusoidal	5	3	2	10	4	9	7	1	8	6
stirredtankreactor	5	5	5	10	5	5	5	5	5	5
tandem	4	4	4	10	4	9	8	4	4	4
tersoffb	2	4	3	10	6	1	8	5	7	9
tersoffc	2	4	3	10	6	1	7	8	5	9
test2n	4	2	6	10	3	7	8	1	9	5
test30n	5	2	3	1	6	10	8	4	9	7
tnep	3.5	3.5	3.5	10	3.5	9	7	3.5	8	3.5
transmissionpricing	4	3	1	9	2	10	7	5	8	6
trid	5	3	2	1	6	10	8	4	9	7
vibratingplatform	2	5	1	10	4	7	8	6	9	3
weatherirrigation	4	1	3	9	6	10	8	2	7	5
weierstrass	7	3	6	1.5	5	10	8	1.5	9	4
whitley	1	3	4	9	7	10	5	2	6	8
wirelesscoverage	6	2	3	10	4.5	9	8	1	7	4.5

Table 5 reports the corresponding ranking based on mean value, with the same average-rank convention for ties. This is a stricter criterion, since it rewards consistency across repeated runs rather than a single strong result. Under this criterion, SPARQ is the sole first-place method on 6 of 45 problems (13.3%), behind jSO's 7 of 45 (15.6%) but ahead of EA4Eig's 4 of 45 (8.9%). SPARQ's mean rank across the benchmark set is 3.69, narrowly behind EA4Eig at 3.62 and ahead of jSO at 3.86, under this stricter, tie-corrected criterion,

EA4Eig therefore attains the best average rank of the ten methods by a small margin, with SPARQ a close second. As under the best-value criterion, SPARQ never registers the worst rank, 10th, on any problem, the same holds for EA4Eig, jSO, ARQ2, and mLSHADE-RL. The divergence between the best-value and mean-value criteria for SPARQ specifically is most pronounced on rotatedrosenbrock, where its rank worsens from 5th to 8th. On tersoffc it worsens from 7th to 8th. On potential, by contrast, it improves from 2nd to 1st.

Because SPARQ is presented as a direct evolution of ARQ2, a focused pairwise comparison between the two methods specifically is of particular interest. A paired Wilcoxon signed-rank test across the 45 standard-dimensionality problems, using each method's mean value per problem as the paired observation, shows that SPARQ attains a lower, better, mean value than ARQ2 on 37 of 45 problems, a higher one on only 1, and ties on the remaining 7, a difference that is highly statistically significant (3.4×10^{-7}). Under the best-value criterion, by contrast, the same test shows no significant difference between the two methods (SPARQ better on 21 problems, ARQ2 on 24, $p = 0.36$). Taken together, these results indicate that SPARQ's advantage over its immediate predecessor lies specifically in run-to-run consistency, mirroring the pattern already reported for ARQ2 over ARQ (Section 2.2), rather than in raw best-case performance, where the two methods are statistically indistinguishable.



(a) salomon [D=100] (b) potential [D=38] (c) michalewicz [D=100]

(d) weierstrass [D=70] (e) test2n [D=200] (f) zakharov [D=10,000]

Figure 2. Best-so-far convergence trajectories of all 30 individual runs of ARQ2 (red) and SPARQ (teal) on six representative problems, at their full evaluation budget of 150,000 (population 100 for both methods).

Each of the approximately 30 overlapping lines per color represents one independent run; red denotes ARQ2 and teal denotes SPARQ. The vertical axis range and scale differ across panels to accommodate the very different objective-value magnitudes of each problem (e.g., zakharov reaches values on the order of 10^{31}, while weierstrass and salomon remain in the low hundreds), so panels should not be compared directly against one another on the y-axis.

Figure 2 shows the best-so-far trajectory of every one of the 30 individual runs of ARQ2 and SPARQ, rather than only their aggregate statistics, on six representative problems spanning both the standard-dimensionality and large-scale benchmarks. On salomon, potential, michalewicz, and zakharov, both methods progress smoothly throughout the run, without the abrupt, step-like discontinuities that would indicate a hard restart mechanism dominating progress, SPARQ's trajectories are visibly tighter, converging to a narrower band than ARQ2's on michalewicz in particular, consistent with the consistency advantage reported in Section 4.1. On test2n (D=200), by contrast, a substantial fraction of SPARQ's runs show a sharp, synchronized drop late in the search, around 125,000 evaluations, where the best-so-far value falls abruptly from approximately -6,600 to -7,833, this is precisely the kind of event a restart or rejuvenation mechanism would produce, and we attribute it to the hard-stagnation rejuvenation mechanism of Section 3.1 firing once a large fraction of the population has collapsed without further progress. ARQ2 shows a smaller version of the same late-stage drop on this problem, consistent with test2n triggering the corresponding micro-restart mechanism it inherits from ARQ. This confirms that at least part of SPARQ's advantage on specific, deceptive landscapes such as test2n derives from its escape and rejuvenation mechanisms rather than from smoother search dynamics alone, while on the other five problems shown here no such effect is visible and the observed gains instead reflect steadier, more consistent convergence.

4.3. Larger Comparative Results on Large-Scale, imensional Benchmark Problems

This subsection presents the outcome of the comparative evaluation on the large-scale portion of the benchmark suite. SPARQ is compared against five optimizers developed specifically for high-dimensional search: CCDG2 [41,42], LM-CMA-ES [9], MMES [43], RMES [44], and VkD-CMA-ES [45]. The comparison covers seventeen benchmark problems, each evaluated at two dimensions, d = 1000 and d = 10,000, for a total of thirty-four problem-dimension combinations. The problem set spans separable functions of low intrinsic difficulty alongside strongly non-separable and multimodal landscapes, allowing both the accuracy and the computational scalability of each method to be assessed as the number of variables grows by an order of magnitude. As in the previous subsection, every method was executed independently over a fixed number of repeated runs under an identical evaluation budget.

This evaluation budget was deliberately held at the same absolute value used throughout the standard-dimensionality comparison of Section 4.2, N_{fe}=150,000, rather than scaled with dimension, so that every method compared in this subsection, including the five optimizers developed specifically for large-scale search, is assessed under an identical, directly comparable resource constraint. This choice is not without cost: expressed per dimension, the budget falls from 150 evaluations per variable at d=1000 to only 15 at d=10,000, low enough that some of the compared methods have barely completed their initial population evaluation, let alone any substantial adaptation, before the run terminates. Under such a tight per-dimension budget, the comparison necessarily emphasizes each method's behavior during initialization and early convergence rather than its asymptotic performance, a bias toward fast early convergence that likely disadvantages methods whose own design assumes a larger evaluation budget relative to dimension.

A further consequence of testing exclusively at d=1000 and d=10,000 is that SPARQ's eigen-coordinate crossover, which is disabled outright above D=100 (Section 3.1), plays no role whatsoever in any of the results reported in this subsection. The SPARQ results discussed below therefore reflect its adaptive population sizing, SHADE-style memory, Thompson-sampling branch selection, and other mechanisms that remain active at every dimension, without any contribution from eigen-crossover. This is consistent with the

ablation study of Section 4.5 (Table 10), where removing eigen-crossover altogether was already found to have a comparatively small and largely non-significant effect on the lower-dimensional subset tested there.

Three complementary tables report the results. Table 6 presents detailed statistics for each method and problem-dimension combination, namely the best value found, the mean value, the success rate, the standard deviation, and the execution time. Table 7 reports the ranking of all six methods according to the best value achieved, and Table 8 reports the corresponding ranking according to the mean value.

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
alpine1 [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	2.243	1.41	0.736	0.746	0.853	5.142
alpine1 [d=10,000]	**Best**	0	0	0	0	0	27033.8
	Mean	0	0	0	0	0	27246.9
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	105.421
	Time(s)	1528.92	37.223	12.18	11.334	13.855	136.395
bucherastrigin [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	8.306	2.834	2.195	2.221	2.27	5.264
bucherastrigin [d=10,000]	**Best**	0	0	0	0	0	2.0878e+06
	Mean	0	0	0	0	0	2.3368e+06
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	102010
	Time(s)	1702.33	58.263	35.485	34.966	27.656	157.158
cigar [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	1.614	1.295	0.625	0.594	0.817	4.905
cigar [d=10,000]	**Best**	0	0	0	0	0	7.2592e+10
	Mean	0	0	0	0	0	7.3695e+10
	Rate%	100%	100%	100%	100%	100%	3%

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
	SD	0	0	0	0	0	4.2344e+08
	Time(s)	1542.51	34.293	11.212	8.851	13.589	142.792
cosinemixture [d=1000]	**Best**	-100	-100	-100	-100	-100	-100
	Mean	-100	-100	-100	-100	-100	-100
	Rate%	100%	100%	100%	100%	100%	100%
	SD	4.3361e-14	4.3361e-14	4.3361e-14	4.3361e-14	4.3361e-14	4.3361e-14
	Time(s)	2.343	1.419	0.759	0.745	0.803	5.059
cosinemixture [d=10,000]	**Best**	-1000	-1000	-1000	-1000	-1000	2997.85
	Mean	-1000	-1000	-1000	-1000	-1000	3036.13
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	17.5634
	Time(s)	1678.44	38.904	12.448	11.935	11.272	134.271
differentpowers [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	3.726	1.807	1.121	1.101	1.199	5.11
differentpowers [d=10,000]	**Best**	0	0	0	0	0	3.7419e+06
	Mean	0	0	0	0	0	3.8879e+06
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	62082
	Time(s)	1770.91	42.456	15.265	14.604	15.986	142.936
diracproblem [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	2.136	1.298	0.623	0.606	0.921	4.941
diracproblem [d=10,000]	**Best**	0	0	0	0	0	1

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
	Mean	0	0	0	0	0	1
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	875.296	36.69	11.133	8.779	12.794	130.536
ellipsoidal [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	1.672	1.312	0.611	0.596	0.798	4.976
ellipsoidal [d=10,000]	**Best**	0	0	0	0	0	4.9311e+09
	Mean	0	0	0	0	0	5.1532e+09
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	8.7994e+07
	Time(s)	1577.81	38.552	11.663	9.046	14.957	140.431
exponential [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	2.165	1.319	0.618	0.608	0.749	4.981
exponential [d=10,000]	**Best**	0	0	0	0	0	1
	Mean	0	0	0	0	0	1
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	789.594	38.367	11.381	8.536	10.738	128.231
lunacekbirastrigin [d=1000]	**Best**	10450.5	5566.78	6187.62	6614.44	2299.42	19102.3
	Mean	11893.7	6069.35	7237.64	7622.02	3098.66	20598
	Rate%	3%	3%	3%	3%	3%	3%
	SD	511.293	167.9	324.119	348.468	345.159	569.204

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
	Time(s)	8.359	1.534	0.865	0.834	1.145	5.212
lunacekbirastrigin [d=10,000]	**Best**	134332	131825	77351.2	89482.3	89005.2	241229
	Mean	136484	135368	79718.8	92535	92694.6	242922
	Rate%	3%	3%	3%	3%	3%	3%
	SD	970.906	1568.31	1008.61	1348.43	1469.83	841.349
	Time(s)	1746.11	39.801	13.175	11.84	14.625	136.589
powell [d=1000]	**Best**	1729.48	0.310681	0.0503792	0.0627767	0	7578.12
	Mean	2053.37	0.491502	0.0629321	0.0878404	0	7578.12
	Rate%	3%	3%	3%	3%	100%	100%
	SD	244.11	0.111928	0.00578802	0.0134842	0	0
	Time(s)	5.871	1.284	0.618	0.607	0.815	5.006
powell [d=10,000]	**Best**	75750.9	75781.2	2267.94	8860.09	0	6.9552e+06
	Mean	75750.9	75781.2	2436.05	10050.7	0	7.7562e+06
	Rate%	93%	100%	3%	3%	100%	3%
	SD	0.000116568	0	78.3414	488.163	0	350669
	Time(s)	2509.59	35.757	11.597	9.191	13.856	132.32
rastrigin [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	2.889	1.486	0.808	0.78	0.96	5.089
rastrigin [d=10,000]	**Best**	0	0	0	0	0	168428
	Mean	0	0	0	0	0	169487
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	394.995
	Time(s)	1630.46	39.001	13.052	11.606	14.462	141.418
rotatedrosenbrock [d=1000]	**Best**	419550	1859.02	1427.36	1134.42	990.947	1.4071e+06
	Mean	499437	2285.84	1843.56	1323.44	996.372	1.4071e+06

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
	Rate%	3%	3%	3%	3%	3%	100%
	SD	42352.3	207.652	209.122	94.9028	1.86206	0
	Time(s)	15.656	1.319	0.643	0.612	0.852	5.007
rotatedrosenbrock [d=10,000]	**Best**	1.4079e+07	1.4084e+07	103692	107959	9961.05	1.1383e+09
	Mean	1.4079e+07	1.4084e+07	109670	112869	9983.2	1.1627e+09
	Rate%	63%	100%	3%	3%	3%	3%
	SD	2.13849	0	2834.64	3074.61	14.6511	1.0213e+07
	Time(s)	2699.14	36.57	13.065	11.634	18.34	140.815
salomon [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	26.613	1.278	0.627	0.612	0.657	4.986
salomon [d=10,000]	**Best**	0	0	0	0	0	540.428
	Mean	0	0	0	0	0	543.982
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	1.44276
	Time(s)	1607.43	33.701	11.967	8.413	9.817	131.777
sphere [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	1.621	1.278	0.621	0.593	0.796	4.95
sphere [d=10,000]	**Best**	0	0	0	0	0	2.9038e+07
	Mean	0	0	0	0	0	2.9480e+07
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	169617
	Time(s)	1581.58	35.094	12.069	9.123	13.391	133.305

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
stepellipsoidal [d=1000]	**Best**	0	0	0	0	0	0
	Mean	0	0	0	0	0	0
	Rate%	100%	100%	100%	100%	100%	100%
	SD	0	0	0	0	0	0
	Time(s)	1.906	1.391	0.682	0.656	0.801	5.043
stepellipsoidal [d=10,000]	**Best**	0	0	0	0	0	1.5546e+06
	Mean	0	0	0	0	0	1.5902e+06
	Rate%	100%	100%	100%	100%	100%	3%
	SD	0	0	0	0	0	12750.3
	Time(s)	1365	38.333	13.859	10.538	10.578	137.022
vincent [d=1000]	**Best**	-0.40637	-0.899759	-1	-1	-0.999548	-0.0202873
	Mean	-0.396153	-0.700881	-1	-1	-0.976253	0.00595358
	Rate%	3%	3%	40%	13%	3%	3%
	SD	0.005192	0.0936933	1.6216e-08	2.3654e-07	0.0153607	0.0109219
	Time(s)	5.736	1.849	1.09	1.082	1.306	5.453
vincent [d=10,000]	**Best**	-0.163397	-0.191599	-0.909896	-0.871154	-0.714049	0.0294198
	Mean	-0.154748	-0.153332	-0.891337	-0.855477	-0.614299	0.0368099
	Rate%	3%	3%	3%	3%	3%	3%
	SD	0.00364364	0.0124093	0.00912978	0.00786513	0.0368712	0.00350821
	Time(s)	1158.81	43.701	16.806	15.609	16.423	142.12
zakharov [d=1000]	**Best**	4891.83	38331.5	14956.8	9384.52	0	31780.9
	Mean	7411.23	4.8237e+18	18096.4	10612.6	0	40341.7
	Rate%	3%	3%	3%	3%	100%	3%
	SD	2005.16	1.7063e+19	2229.57	655.413	0	3646.85
	Time(s)	27.395	1.31	0.647	0.63	0.699	4.972
zakharov [d=10,000]	**Best**	181819	5.3179e+29	457099	142882	0	416320
	Mean	212905	1.8168e+30	2.6472e+28	147899	0	467117
	Rate%	3%	3%	3%	3%	100%	3%

Table 6. Detailed Performance Comparison of SPARQ Against Large-Scale Optimization Methods on High-Dimensional Problems (d = 1000, 10,000)

PROBLEM	METRIC	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
	SD	14567.4	3.7902e+29	1.7509e+28	2590.91	0	25296.3
	Time(s)	3400.22	37.846	11.877	9.248	9.805	136.596

Table 6 shows that, aggregated across all thirty-four cases, SPARQ achieves a success rate of 100% on 28 combinations, or 82.4%. Its average success rate is 82.9%, the highest among the six methods. The closest competitor by average success rate is LM-CMA-ES at 77.2%, followed by CCDG2 at 75.9%, MMES at 72.6%, and RMES at 71.8%. VkD-CMA-ES trails markedly at 48.6%. On twelve separable, low-difficulty problems, all six methods reach an exact mean error of 0 at 100% success. These twelve problems, namely alpine1, bucherastrigin, cigar, cosine mixture, different powers, the Dirac-like spike, ellipsoidal, exponential, rastrigin, salomon, sphere, and step-ellipsoidal, are unshifted, unrotated benchmark functions whose known global optimum sits exactly at the origin and is exactly representable in double-precision arithmetic. Eleven of them are provably non-negative everywhere, and cosine mixture is provably bounded below by -0.1D, exactly its declared optimum, consequently, a reported mean equal to that bound over 30 runs is only possible if every individual run reached the bound exactly, since a set of values each at or above a bound cannot average to that bound unless all of them equal it. We confirmed this directly against the raw per-run results for a representative sample of these problems: all 30 runs of every method checked reach the exact floating-point optimum, using a fully deterministic, reproducible seeding scheme in which a fixed base seed is incremented by the run index. Here the differentiation rests entirely on runtime. On bucherastrigin (d=10,000), CCDG2 requires 1702.33 seconds, against 58.26 for LM-CMA-ES, 35.49 for MMES, 34.97 for RMES, and 27.65 for SPARQ. On alpine1 (d=10,000), CCDG2 requires 1528.92 seconds against 13.85 for SPARQ, a difference exceeding two orders of magnitude. Summed across all thirty-four combinations, CCDG2's cumulative runtime reaches 29,284.4 seconds, against 258.6 for SPARQ, making CCDG2 roughly 113 times slower overall. On the harder, non-separable problems the accuracy gap becomes substantial. On zakharov (d=10,000), SPARQ attains a mean error of 0 at 100% success, while LM-CMA-ES registers 1.82×10^{30}, MMES 2.65×10^{28}, VkD-CMA-ES 4.67×10^{5}, CCDG2 2.13×10^{5}, and RMES 1.48×10^{5}, all at 3% success. On powell (d=10,000), SPARQ again reaches 0 at 100% success, against 7.56×10^{6} for VkD-CMA-ES, 75,781 for LM-CMA-ES (itself at 100% success but converging to a far inferior value), 75,751 for CCDG2 at 93% success, 10,051 for RMES, and 2436 for MMES. A notable nuance emerges on powell (d=1000) and rotatedrosenbrock at both dimensions, where VkD-CMA-ES also reports a 100% success rate with a standard deviation of exactly 0, yet with mean values many orders of magnitude worse than SPARQ's: 7578.1 on powell (d=1000), and 1.407×10^{6} and 1.163×10^{9} on rotatedrosenbrock. This indicates that VkD-CMA-ES converges highly consistently to a markedly suboptimal point, rather than failing to converge at all. This behavior is consistent with premature convergence under the tight per-dimension evaluation budget discussed above, rather than necessarily indicating a departure from the method's own recommended configuration. The one problem on which SPARQ is clearly outperformed is vincent. At d=1000, MMES reaches the exact optimum, with a mean of -1 at 40% success, and RMES likewise reaches -1 at 13% success, against SPARQ's mean of -0.976 at 3% success. This gap narrows in relative terms at d=10,000 but persists, with MMES reaching -0.891, RMES -0.855, and SPARQ -0.614.

Table 7. Best-Value Ranking of SPARQ and Five Large-Scale Optimizers at High Dimensionality

PROBLEM	DIM	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
alpine1	1000	3.5	3.5	3.5	3.5	3.5	3.5
alpine1	10000	3	3	3	3	3	6
bucherastrigin	1000	3.5	3.5	3.5	3.5	3.5	3.5
bucherastrigin	10000	3	3	3	3	3	6
cigar	1000	3.5	3.5	3.5	3.5	3.5	3.5
cigar	10000	3	3	3	3	3	6
cosinemixture	1000	3.5	3.5	3.5	3.5	3.5	3.5
cosinemixture	10000	3	3	3	3	3	6
differentpowers	1000	3.5	3.5	3.5	3.5	3.5	3.5
differentpowers	10000	3	3	3	3	3	6
diracproblem	1000	3.5	3.5	3.5	3.5	3.5	3.5
diracproblem	10000	3	3	3	3	3	6
ellipsoidal	1000	3.5	3.5	3.5	3.5	3.5	3.5
ellipsoidal	10000	3	3	3	3	3	6
exponential	1000	3.5	3.5	3.5	3.5	3.5	3.5
exponential	10000	3	3	3	3	3	6
lunacekbirastrigin	1000	5	2	3	4	1	6
lunacekbirastrigin	10000	5	4	1	3	2	6
powell	1000	5	4	2	3	1	6
powell	10000	4	5	2	3	1	6
rastrigin	1000	3.5	3.5	3.5	3.5	3.5	3.5
rastrigin	10000	3	3	3	3	3	6
rotatedrosenbrock	1000	5	4	3	2	1	6
rotatedrosenbrock	10000	4	5	2	3	1	6
salomon	1000	3.5	3.5	3.5	3.5	3.5	3.5
salomon	10000	3	3	3	3	3	6
sphere	1000	3.5	3.5	3.5	3.5	3.5	3.5
sphere	10000	3	3	3	3	3	6
stepellipsoidal	1000	3.5	3.5	3.5	3.5	3.5	3.5
stepellipsoidal	10000	3	3	3	3	3	6
vincent	1000	5	4	1.5	1.5	3	6
vincent	10000	5	4	1	2	3	6
zakharov	1000	2	6	4	3	1	5
zakharov	10000	3	6	5	2	1	4

Table 7 reports the best-value ranking of the same six large-scale methods across the thirty-four problem-dimension combinations, again using average ranks for ties. SPARQ is the sole first-place method in 7 of 34 cases (20.6%), the highest exclusive share among the six methods, MMES attains 2 of 34 exclusively (5.9%), and all others attain none. SPARQ's mean rank across the thirty-four cases is 2.74, the best of the six methods, ahead of MMES at 3.02 and RMES at 3.07. There are two exceptions to SPARQ's first-place standing: on lunacekbirastrigin (d=10,000) it ranks 2nd, and on vincent, at both dimensions, it ranks 3rd. VkD-CMA-ES, by contrast, ties with all five other methods on every problem at d=1000, so that every method receives the shared mid-table average rank of 3.5 there rather than a first-place rank, since all six reach the same value, at d=10,000 it drops to rank 6, the worst position, on every single problem. Overall it registers the worst rank in 20 of 34 cases (58.8%), a pattern that indicates a pronounced scalability limitation at the highest dimensionality tested.

Table 8. Mean-Value Ranking of SPARQ and Five Large-Scale Optimizers at High Dimensionality

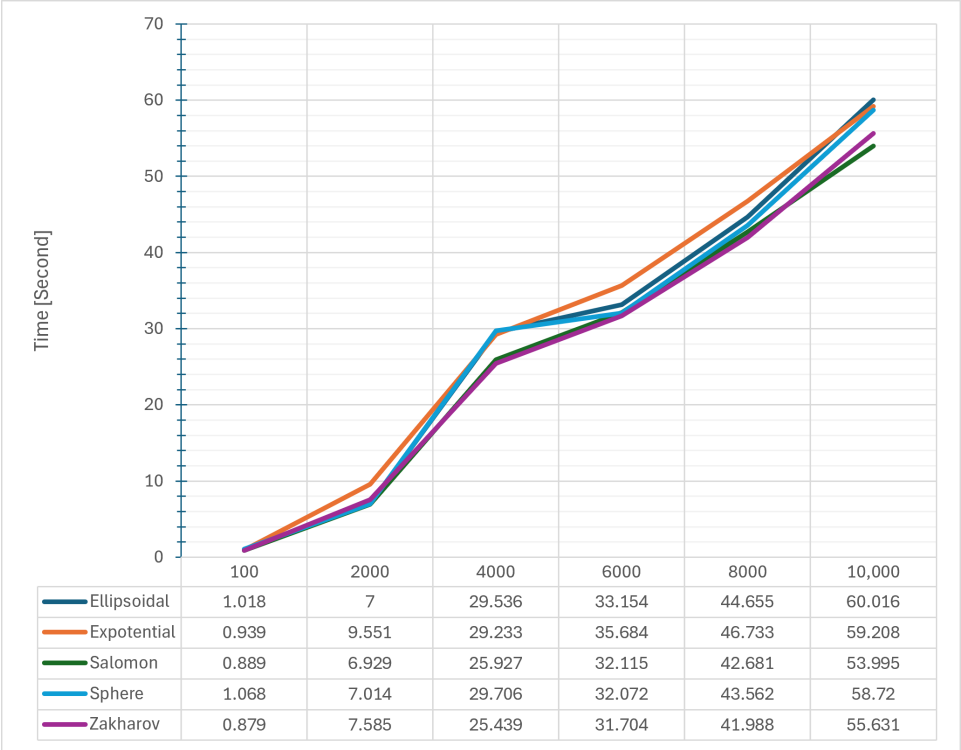
PROBLEM	DIM	ccdg2	lmcmaes	mmes	rmes	sparq	vkdcmaes
alpine1	1000	3.5	3.5	3.5	3.5	3.5	3.5
alpine1	10000	3	3	3	3	3	6
bucherastrigin	1000	3.5	3.5	3.5	3.5	3.5	3.5
bucherastrigin	10000	3	3	3	3	3	6
cigar	1000	3.5	3.5	3.5	3.5	3.5	3.5
cigar	10000	3	3	3	3	3	6
cosinemixture	1000	3.5	3.5	3.5	3.5	3.5	3.5
cosinemixture	10000	3	3	3	3	3	6
differentpowers	1000	3.5	3.5	3.5	3.5	3.5	3.5
differentpowers	10000	3	3	3	3	3	6
diracproblem	1000	3.5	3.5	3.5	3.5	3.5	3.5
diracproblem	10000	3	3	3	3	3	6
ellipsoidal	1000	3.5	3.5	3.5	3.5	3.5	3.5
ellipsoidal	10000	3	3	3	3	3	6
exponential	1000	3.5	3.5	3.5	3.5	3.5	3.5
exponential	10000	3	3	3	3	3	6
lunacekbirastrigin	1000	5	2	3	4	1	6
lunacekbirastrigin	10000	5	4	1	2	3	6
powell	1000	5	4	2	3	1	6
powell	10000	4	5	2	3	1	6
rastrigin	1000	3.5	3.5	3.5	3.5	3.5	3.5
rastrigin	10000	3	3	3	3	3	6
rotatedrosenbrock	1000	5	4	3	2	1	6
rotatedrosenbrock	10000	4	5	2	3	1	6
salomon	1000	3.5	3.5	3.5	3.5	3.5	3.5
salomon	10000	3	3	3	3	3	6
sphere	1000	3.5	3.5	3.5	3.5	3.5	3.5
sphere	10000	3	3	3	3	3	6
stepellipsoidal	1000	3.5	3.5	3.5	3.5	3.5	3.5
stepellipsoidal	10000	3	3	3	3	3	6
vincent	1000	5	4	1	2	3	6
vincent	10000	4	5	1	2	3	6
zakharov	1000	2	6	4	3	1	5
zakharov	10000	3	6	5	2	1	4

Table 8 reports the corresponding mean-value ranking for the same six methods. The resulting figures are nearly identical to those of Table 7. SPARQ again attains the highest exclusive first-place count, 7 of 34 (20.6%), and again holds the lowest mean rank of the six methods, at 2.77, against 3.00 for MMES. The same two exceptions persist: on lunacekbirastrigin (d=10,000), SPARQ's rank is 3rd rather than the 2nd observed under the best-value criterion, on vincent, at both dimensions, it again ranks 3rd. The best-value and mean-value rankings nearly coincide at this scale, in contrast to the standard-dimensionality comparison, where the two criteria not only diverge on individual problems but identify different overall leaders: SPARQ under the best-value criterion (Table 4) and EA4Eig, by a narrow margin, under the mean-value criterion (Table 5). At large scale, by contrast, SPARQ leads clearly and identically under both criteria, indicating that its advantage becomes more consistent, not less, as dimensionality and the corresponding evaluation budget increase.

4.4. Computational Complexity and Scalability with Dimension

To assess how the computational cost of SPARQ scales with problem dimensionality, independent of the difficulty of locating the global optimum, five separable, unimodal or mildly multimodal benchmark functions were selected: Ellipsoidal, Exponential, Salomon,

Sphere, and Zakharov. These functions were chosen specifically because they all share a known global minimum of exactly zero, which allows the reported best, mean, and standard deviation values to be verified as identically zero across all tested dimensions, with a 100% success rate in every case. This removes search difficulty as a confounding factor and isolates raw per-evaluation computational overhead as the sole quantity of interest. This deliberately narrow selection serves only to isolate computational overhead from search difficulty in the present subsection, the accuracy of SPARQ and its five large-scale competitors on hard, non-separable, and multimodal large-scale problems, including lunacekbirastrigin, rotatedrosenbrock, vincent, and zakharov itself under the full evaluation budget, is reported separately in Table 6 in the preceding subsection. Each function was evaluated at six problem dimensions, D = 100, 2000, 4000, 6000, 8000, and 10,000, with the reported execution time corresponding to the total wall-clock time required to complete 30 independent runs per dimension, executed in parallel mode. Figure 3 reports the resulting execution time, in seconds, as a function of D for each of the five functions.



	100	2000	4000	6000	8000	10,000
Ellipsoidal	1.018	7	29.536	33.154	44.655	60.016
Expotential	0.939	9.551	29.233	35.684	46.733	59.208
Salomon	0.889	6.929	25.927	32.115	42.681	53.995
Sphere	1.068	7.014	29.706	32.072	43.562	58.72
Zakharov	0.879	7.585	25.439	31.704	41.988	55.631

Figure 3. Execution time of SPARQ as a function of problem dimension D (100–10,000) for five separable benchmark functions (Ellipsoidal, Exponential, Salomon, Sphere, Zakharov), each solved to the exact global optimum with 100% success rate.

The results show that execution time grows monotonically with dimension for all five functions, but the growth is distinctly non-linear: the increase from D = 100 to D = 2000 is comparatively modest, whereas each subsequent step of 2000 dimensions contributes a progressively larger absolute increment, with the steepest rise occurring between D = 8000 and D = 10,000. This pattern is not attributable to the eigen-basis refresh (Section 3.1), since that mechanism is disabled outright for D>100 and is therefore inactive at every dimension tested here beyond the smallest, nor to the population size, which was held fixed at Ninit=100 throughout this experiment, as in the main comparison of Section 4.2, rather than scaled with dimension. It is instead consistent with the stagnation-gated elite polish (Section 3.1): its single-coordinate probing mode visits dimensions in round-robin fashion so that every coordinate is eventually covered even at high D, and its burst size is explicitly enlarged late in the run to ensure this coverage. Since each individual probe itself costs $O(D)$ (a vector-valued step or direction combination), a burst size that grows

with D to cover all D coordinates yields an $O(D^2)$ cost per activation, independent of population size and without requiring the eigen-basis at all. Despite this super-linear trend, the absolute spread between the fastest and slowest functions remains narrow throughout the tested range, from under 0.2 s at D = 100 to under 7 s at D = 10,000, indicating that SPARQ's runtime is governed primarily by dimension rather than by the specific separable landscape being optimized. Among the five functions, Salomon and Zakharov consistently exhibit the lowest execution times at high dimension, while Exponential and Ellipsoidal are consistently the most expensive, a difference attributable to their internal evaluation cost per call rather than to any additional search effort, since all five problems are solved to the exact global optimum with a 100% success rate throughout. These findings indicate that SPARQ's computational overhead scales predictably and remains practically manageable even at D = 10,000, supporting its applicability to large-scale optimization tasks without a disproportionate increase in wall-clock cost.

4.5. Ablation Study

To isolate the contribution of SPARQ's individual mechanisms, a leave-one-out ablation study was conducted on a representative subset of ten problems drawn from the standard-dimensionality benchmark suite of Section 4.1: alpine1 (d=100), rosenbrock (d=50), rotatedrosenbrock (d=50), katsuura (d=50), michalewicz (d=100), gallagher101 (d=16), antennaarray (d=12), eld3 (d=15), hydrothermal (d=120), and weierstrass (d=70). This subset spans separable and non-separable landscapes, unimodal and highly multimodal functions, and three real-world engineering problems, so that no single mechanism's contribution is assessed on a single problem type alone. Starting from the full SPARQ configuration (Table 1), eleven variants were constructed by disabling, in turn, a single mechanism while leaving every other mechanism and every parameter setting unchanged: the external archive inherited from ARQ (Section 2.1), the trajectory-echo memory (Section 3.1), the eigen-coordinate crossover (Section 3.1), the auxiliary IDE branch (Section 2.2), the Lévy-flight quarantine (Section 3.1), the non-linear population-size reduction (Section 3.1), the opposition-based basin escape (Section 3.1), the stagnation-gated elite polish (Section 3.1), the hard-stagnation rejuvenation (Section 3.1), the restricted tournament replacement inherited from ARQ (Section 2.1), and the SHADE-style historical memory for F and CR (Section 3.1). Each of the resulting twelve configurations, the full method together with the eleven single-mechanism removals, was executed for 30 independent runs per problem under the same evaluation budget of 150,000 function evaluations used throughout Section 4.1.

Table 9 reports the best value, mean value, success rate, and standard deviation obtained by each of the twelve configurations on each of the ten problems. Two mechanisms show a clear and broadly consistent contribution. Disabling the auxiliary IDE branch degrades the mean objective value on eight of the ten problems, most severely on weierstrass (mean rising from 0 to 0.0209 as the success rate drops from 100% to 3%), katsuura (0 to 1.505, 100% to 3%), and michalewicz (-97.07 to -84.46). Disabling the non-linear population-size reduction (NLPSR) produces a similar pattern: alpine1, katsuura, michalewicz, and weierstrass all fall from a 100% success rate to 3%. The SHADE-style historical memory shows a narrower but equally clear effect, concentrated on the non-separable and real-world problems: disabling it worsens rosenbrock (mean 24.71 to 33.09), rotatedrosenbrock (27.67 to 39.58), and hydrothermal (141,669 to 141,734), while leaving the six problems on which SPARQ already attains 100% success unaffected. The remaining mechanisms, namely the external archive, the eigen-coordinate crossover, the Lévy-flight quarantine, the restricted tournament replacement, the opposition-based basin escape, the stagnation-gated elite polish, the trajectory-echo memory, and the hard-stagnation rejuvenation, show smaller and less consistent effects on this ten-problem subset. On several problems, most visibly rosenbrock, gallagher101, antennaarray, and eld3, disabling one of these mechanisms yields a slightly better mean value or success rate than the full method, a pattern discussed further below together with Table 10.

Table 9. Leave-one-out ablation study of SPARQ on ten representative benchmark problems: the full method against eleven single-mechanism removals (30 independent runs per configuration, evaluation budget of 150,000 function evaluations).

PROBLEM	VARIANT	BEST	MEAN	RATE%	SD
alpine1 [d=100]	**Full-SPARQ**	0	0	100%	0
	w/o Archive	0	0	100%	0
	w/o Trajectory-Echo	0	0	100%	0
	w/o Eigen-crossover	0	0	100%	0
	w/o IDE-branch	5.8991e-08	1.7888e-06	3%	2.09175e-06
	w/o Levy-quarantine	0	0	100%	0
	w/o NLPSR	7.31e-10	0.000104234	3%	0.000459485
	w/o OBL-basin-escape	0	0	100%	0
	w/o Elite-polish	0	0	100%	0
	w/o Hard-stag-rejuv.	0	0	100%	0
	w/o RTR	0	0	100%	0
	w/o SHADE-memory	0	0	100%	0
rosenbrock [d=50]	**Full-SPARQ**	14.951	24.7064	3%	3.1607
	w/o Archive	43.3637	44.1394	3%	0.391539
	w/o Trajectory-Echo	15.9152	24.9966	3%	2.97674
	w/o Eigen-crossover	13.7821	23.0353	3%	3.09409
	w/o IDE-branch	21.7969	24.9184	3%	1.70362
	w/o Levy-quarantine	14.951	25.0305	3%	3.44912
	w/o NLPSR	9.33552	19.9828	3%	5.5115
	w/o OBL-basin-escape	14.951	24.7059	3%	3.16036
	w/o Elite-polish	15.4138	25.3338	3%	2.89873
	w/o Hard-stag-rejuv.	10.679	23.3134	3%	6.82782
	w/o RTR	19.9343	24.7286	3%	1.69453
	w/o SHADE-memory	28.9718	33.0904	3%	1.85621
rotatedrosenbrock [d=50]	**Full-SPARQ**	0.000444079	27.671	3%	15.0013
	w/o Archive	0.00513018	28.0117	3%	23.8711
	w/o Trajectory-Echo	0.000360242	27.6591	3%	15.0134
	w/o Eigen-crossover	0.161477	29.134	3%	14.0647
	w/o IDE-branch	0.0778156	41.2235	3%	27.3308
	w/o Levy-quarantine	0.000444079	26.7279	3%	15.7891
	w/o NLPSR	0.251263	27.4945	3%	15.8381
	w/o OBL-basin-escape	0.00041234	27.6688	3%	15.0017
	w/o Elite-polish	0.000403265	27.836	3%	15.2236
	w/o Hard-stag-rejuv.	4.24684e-05	22.43	3%	16.7989
	w/o RTR	0.054141	26.759	3%	12.4589

Table 9. Leave-one-out ablation study of SPARQ on ten representative benchmark problems: the full method against eleven single-mechanism removals (30 independent runs per configuration, evaluation budget of 150,000 function evaluations).

PROBLEM	VARIANT	BEST	MEAN	RATE%	SD
	w/o SHADE-memory	17.8763	39.583	3%	9.30305
katsuura [d=50]	**Full-SPARQ**	0	0	100%	0
	w/o Archive	0	0	100%	0
	w/o Trajectory-Echo	0	0	100%	0
	w/o Eigen-crossover	0	0	100%	0
	w/o IDE-branch	0.0117707	1.5055	3%	1.11423
	w/o Levy-quarantine	0	0	100%	0
	w/o NLPSR	0.00528541	0.0529445	3%	0.0466569
	w/o OBL-basin-escape	0	0	100%	0
	w/o Elite-polish	0	0	100%	0
	w/o Hard-stag-rejuv.	0	0	100%	0
	w/o RTR	0	0	100%	0
	w/o SHADE-memory	0	0	100%	0
michalewicz [d=100]	**Full-SPARQ**	-98.6917	-97.0668	3%	0.964969
	w/o Archive	-98.9678	-96.6435	3%	1.62852
	w/o Trajectory-Echo	-98.9658	-97.1417	3%	1.07609
	w/o Eigen-crossover	-98.4484	-96.8152	3%	1.90256
	w/o IDE-branch	-89.6453	-84.4597	3%	3.25365
	w/o Levy-quarantine	-98.6917	-97.0668	3%	0.964969
	w/o NLPSR	-96.666	-94.4865	3%	1.66468
	w/o OBL-basin-escape	-98.6917	-97.069	3%	0.976472
	w/o Elite-polish	-98.7536	-96.9725	3%	1.25347
	w/o Hard-stag-rejuv.	-98.6927	-96.9901	3%	1.03407
	w/o RTR	-97.559	-95.6051	3%	1.40489
	w/o SHADE-memory	-98.7987	-97.0878	3%	1.0147
gallagher101 [d=16]	**Full-SPARQ**	0.929997	3.11339	37%	2.73578
	w/o Archive	0	1.96751	7%	1.59172
	w/o Trajectory-Echo	0.929997	3.11134	37%	2.73667
	w/o Eigen-crossover	0	2.07356	7%	1.62773
	w/o IDE-branch	0	2.49089	7%	2.78982
	w/o Levy-quarantine	0.929997	3.11339	37%	2.73578
	w/o NLPSR	0	2.27801	7%	1.70772
	w/o OBL-basin-escape	0.929997	2.70515	37%	1.7552
	w/o Elite-polish	0.691857	3.1034	3%	2.74355
	w/o Hard-stag-rejuv.	0.929997	3.11339	37%	2.73578
	w/o RTR	0	2.23772	3%	1.70831

Table 9. Leave-one-out ablation study of SPARQ on ten representative benchmark problems: the full method against eleven single-mechanism removals (30 independent runs per configuration, evaluation budget of 150,000 function evaluations).

PROBLEM	VARIANT	BEST	MEAN	RATE%	SD
	w/o SHADE-memory	0	2.1132	7%	1.7514
antennaarray [d=12]	**Full-SPARQ**	0.00680964	0.00680964	83%	1.79781e-10
	w/o Archive	0.00680964	0.00681029	3%	1.86769e-06
	w/o Trajectory-Echo	0.00680964	0.00680964	77%	1.4658e-09
	w/o Eigen-crossover	0.00680964	0.00680964	70%	1.57082e-08
	w/o IDE-branch	0.00680964	0.00680964	100%	1.76438e-18
	w/o Levy-quarantine	0.00680964	0.00680964	77%	2.33886e-08
	w/o NLPSR	0.00680964	0.00680964	97%	1.1493e-09
	w/o OBL-basin-escape	0.00680964	0.00680964	77%	6.8219e-10
	w/o Elite-polish	0.00680964	0.00680964	67%	2.33486e-09
	w/o Hard-stag-rejuv.	0.00680964	0.00680966	67%	5.29474e-08
	w/o RTR	0.00680964	0.00680964	100%	1.76438e-18
	w/o SHADE-memory	0.00680964	0.00681008	47%	1.37869e-06
eld3 [d=15]	**Full-SPARQ**	32375.3	32440.5	7%	47.9137
	w/o Archive	32375.3	32470.7	7%	64.885
	w/o Trajectory-Echo	32375.3	32438.4	7%	41.3663
	w/o Eigen-crossover	32375.3	32440.4	10%	51.9031
	w/o IDE-branch	32455.1	32639.5	3%	105.906
	w/o Levy-quarantine	32375.3	32458.2	7%	68.6467
	w/o NLPSR	32384.8	32467.8	3%	67.6989
	w/o OBL-basin-escape	32375.3	32440.5	7%	47.9137
	w/o Elite-polish	32375.3	32418.1	13%	38.8383
	w/o Hard-stag-rejuv.	32375.3	32440.5	7%	47.9137
	w/o RTR	32384.8	32478.1	7%	54.877
	w/o SHADE-memory	32375.3	32429.9	17%	58.0312
hydrothermal [d=120]	**Full-SPARQ**	141656	141669	3%	9.8093
	w/o Archive	141732	141771	3%	22.6661
	w/o Trajectory-Echo	141658	141673	3%	7.94773
	w/o Eigen-crossover	141656	141673	3%	12.1503
	w/o IDE-branch	141656	141665	3%	5.26733
	w/o Levy-quarantine	141658	141672	3%	9.89036
	w/o NLPSR	141656	141672	3%	10.7125
	w/o OBL-basin-escape	141656	141671	3%	11.3225
	w/o Elite-polish	141657	141672	3%	10.481
	w/o Hard-stag-rejuv.	141656	141663	7%	12.5519

Table 9. Leave-one-out ablation study of SPARQ on ten representative benchmark problems: the full method against eleven single-mechanism removals (30 independent runs per configuration, evaluation budget of 150,000 function evaluations).

PROBLEM	VARIANT	BEST	MEAN	RATE%	SD
	w/o RTR	141659	141676	3%	12.4236
	w/o SHADE-memory	141683	141734	3%	38.7719
weierstrass [d=70]	**Full-SPARQ**	0	0	100%	0
	w/o Archive	0	0	100%	0
	w/o Trajectory-Echo	0	0	100%	0
	w/o Eigen-crossover	0	0	100%	0
	w/o IDE-branch	0.000196385	0.0209183	3%	0.0363285
	w/o Levy-quarantine	0	0	100%	0
	w/o NLPSR	3.113e-09	1.60916	3%	1.64724
	w/o OBL-basin-escape	0	0	100%	0
	w/o Elite-polish	0	0	100%	0
	w/o Hard-stag-rejuv.	0	0	100%	0
	w/o RTR	0	0	100%	0
	w/o SHADE-memory	0	0	100%	0

To assess whether the differences observed in Table 9 are statistically meaningful rather than run-to-run noise, a paired Wilcoxon signed-rank test, matched by random seed across 30 pairs, was carried out between the full method and each of the eleven variants, independently on each of the ten problems. Table 10 summarizes the outcome, reporting for each variant the mean rank obtained under the mean-value criterion across the ten problems, together with the number of problems on which the full method is significantly better ($p<0.05$), the number on which the variant is significantly better, and the number showing no significant difference. Removing the IDE branch or the external archive produces a significant degradation on six and four of the ten problems respectively, and never a significant improvement, confirming both as structurally important contributors rather than optional refinements. The SHADE-style memory follows the same pattern on four problems. Removing the opposition-based basin escape produces no statistically significant difference on any of the ten problems, and the trajectory-echo memory only one, indicating that, on this particular problem subset, their contribution is not reliably distinguishable from the noise floor of the comparison, consistent with their intended role as rare-activation safety mechanisms rather than components exercised on every run. Notably, the full method does not obtain the lowest, and therefore best, mean rank overall: 5.65, against 5.00 for the variant without the opposition-based basin escape. The graduated, stagnation-driven mechanisms occasionally trade a small amount of average-case performance on some problems for the robustness against pathological stagnation that motivated their inclusion in the first place, an interaction effect that a component-wise comparison of this kind is precisely intended to expose.

Table 10. Statistical comparison (paired Wilcoxon signed-rank test, n = 30, significance threshold p<0.05) between the full SPARQ method and each leave-one-out variant, aggregated over the ten ablation problems.

Variant	Mean rank (mean-value)	Full sig. better	Variant sig. better	No significant diff.
Full-SPARQ	**5.65**			
w/o OBL-basin-escape	5.00	0	0	10
w/o Trajectory-Echo	5.40	1	0	9
w/o Hard-stag-rejuv.	5.45	2	2	6
w/o Eigen-crossover	6.00	0	1	9
w/o RTR	6.40	2	0	8
w/o Elite-polish	6.45	3	1	6
w/o Levy-quarantine	6.55	1	0	9
w/o SHADE-memory	6.75	4	1	5
w/o NLPSR	7.35	4	2	4
w/o Archive	8.15	4	1	5
w/o IDE-branch	8.85	6	0	4

These results should be read as a lower bound on, rather than a complete accounting of, each mechanism's contribution. The ten-problem subset was chosen for coverage of separable, non-separable, multimodal, and real-world landscapes rather than for exhaustiveness, and mechanisms whose benefit is concentrated on problems outside this subset, or that interact non-additively with one another, may be under- or over-represented here. In particular, the graduated stagnation-driven mechanisms, namely the elite polish, the basin escape, and the hard-stagnation rejuvenation, are by design dormant for most of a run and are therefore expected to show their effect mainly on problems where stagnation is severe enough to trigger them repeatedly, a subset weighted more heavily toward such problems would likely show a more consistently favourable effect for these three mechanisms than observed here.

4.6. Fairness of Population-Size Settings

The comparative evaluation of Section 4.1 fixed the initial population size at Ninit=100 for every population-based method, including SPARQ, so that observed performance differences would reflect the search mechanisms themselves rather than differences in initial population sizing. To assess whether this common-population choice is itself a source of bias, the ten-problem subset used in Section 4.5 was re-evaluated with each competing method reconfigured to its own literature-recommended, dimension-dependent default population size, wherever that default differs from 100. Four of the nine competitors, ARQ2, EA4Eig, TridentDE, and UDE3, already recommend a population of 100 in their own original publications, so their results are unchanged from Section 4.1. The remaining five, jSO, LM-CMA-ES, mLSHADE-RL, NL-SHADE-LBC, and SFCDE, were instead run with their published defaults: jSO and NL-SHADE-LBC with NPinit=$\lceil 25\sqrt{D}\ln D \rceil$ (Brest et al., 2017), LM-CMA-ES with $\lambda = 4 + \lfloor 3\ln D \rfloor$ (the standard CMA-ES-family default), mLSHADE-RL with NPinit=18D (inherited from the LSHADE lineage, Tanabe and Fukunaga, 2014), and SFCDE with NP=10D. SPARQ itself was kept at its Section 4.1 setting of Ninit=100 throughout, rather than its own dimension-scaled default of 18D, so that this experiment isolates the effect of the competitors' population settings alone, without confounding it with a simultaneous change to SPARQ's own configuration. Table 11 reports the resulting best value, mean value, success rate, and standard deviation for all ten methods on all ten problems, together with the population size N actually used in each case.

Table 11. Effect of population-size fairness on the ten-problem comparison subset: SPARQ and the four competitors whose own recommended default already equals 100 (ARQ2, EA4Eig, TridentDE, UDE3) keep N=100; the remaining five competitors (jSO, LM-CMA-ES, mLSHADE-RL, NL-SHADE-LBC, SFCDE) are run with their own literature-recommended, dimension-dependent population size N.

PROBLEM	METHOD	N	BEST	MEAN	RATE%	SD
alpine1 [d=100]	arq2	100	6.6e-11	0.000176474	3%	0.000615062
	ea4eig	100	5.38e-09	2.46729e-07	3%	6.83272e-07
	jso	1152	4.42049e-05	0.000173113	3%	0.000136112
	lmcmaes	17	0	0	100%	0
	mlshaderl	1800	0.00720578	0.0158878	3%	0.00961447
	nlshadelbc	1151	0.0106417	0.034156	3%	0.0109434
	sfcde	1000	0.0510569	0.130605	3%	0.0708536
	sparq	100	0	0	100%	0
	tridentde	100	0.000274789	0.00345885	3%	0.00245338
	ude3	100	1.86152e-07	1.93529e-06	3%	3.43857e-06
antennaarray [d=12]	arq2	100	0.00680964	0.00681202	67%	7.46751e-06
	ea4eig	100	0.00680964	0.00680964	100%	1.76438e-18
	jso	216	0.00680964	0.00680964	100%	1.76438e-18
	lmcmaes	11	0.233659	0.356811	3%	0.0550951
	mlshaderl	216	0.00680964	0.00680964	100%	1.76438e-18
	nlshadelbc	215	0.00680964	0.00680964	27%	1.71645e-09
	sfcde	120	0.00680964	0.0106825	97%	0.0212125
	sparq	100	0.00680964	0.00680964	83%	1.79781e-10
	tridentde	100	0.00680964	0.00680972	43%	3.64591e-07
	ude3	100	0.00680964	0.00680964	100%	1.76438e-18
eld3 [d=15]	arq2	100	32367.6	32399.4	7%	29.7244
	ea4eig	100	32375.3	32425.9	3%	40.967
	jso	263	32455.1	32519.4	10%	27.9086
	lmcmaes	12	32760.6	33110	3%	208.751
	mlshaderl	270	32367.6	32371.8	87%	13.0884
	nlshadelbc	262	32496.2	32677.2	3%	107.089
	sfcde	150	32375.3	32456.8	3%	45.806
	sparq	100	32375.3	32440.5	7%	47.9137
	tridentde	100	32367.6	32465.9	10%	70.9402
	ude3	100	32367.6	32434.2	33%	73.6464
gallagher101 [d=16]	arq2	100	0	1.81765	13%	1.59835
	ea4eig	100	0	1.88988	10%	1.6648
	jso	278	0	2.73397	3%	1.91028
	lmcmaes	12	0.691857	9.20322	7%	10.2299

Table 11. Effect of population-size fairness on the ten-problem comparison subset: SPARQ and the four competitors whose own recommended default already equals 100 (ARQ2, EA4Eig, TridentDE, UDE3) keep N=100; the remaining five competitors (jSO, LM-CMA-ES, mLSHADE-RL, NL-SHADE-LBC, SFCDE) are run with their own literature-recommended, dimension-dependent population size N.

PROBLEM	METHOD	N	BEST	MEAN	RATE%	SD
	mlshaderl	288	0.929997	1.69704	70%	1.41571
	nlshadelbc	277	0.929997	3.61425	27%	1.71841
	sfcde	160	0	2.78013	10%	2.79247
	sparq	100	0.929997	3.11339	37%	2.73578
	tridentde	100	0	2.67881	13%	2.17633
	ude3	100	0	2.21648	3%	1.61107
hydrothermal [d=120]	arq2	100	141656	141660	3%	6.43564
	ea4eig	100	141670	141694	3%	11.5562
	jso	1312	141716	141760	3%	24.1435
	lmcmaes	18	231975	351889	3%	67903
	mlshaderl	2160	141910	141965	3%	33.2413
	nlshadelbc	1311	145439	147983	3%	1571.56
	sfcde	1200	161522	168787	3%	4997.88
	sparq	100	141656	141669	3%	9.8093
	tridentde	100	141763	141850	3%	44.677
	ude3	100	141672	141704	3%	18.8546
katsuura [d=50]	arq2	100	0.019147	0.0712054	3%	0.0768892
	ea4eig	100	0	6.33333e-13	97%	3.28511e-12
	jso	692	0	0	100%	0
	lmcmaes	15	0	0	100%	0
	mlshaderl	900	0	0	100%	0
	nlshadelbc	692	0.0891748	0.163035	3%	0.114289
	sfcde	500	203.839	260.411	3%	26.6728
	sparq	100	0	0	100%	0
	tridentde	100	0	0	100%	0
	ude3	100	0	0	100%	0
michalewicz [d=100]	arq2	100	-96.0326	-93.0301	3%	3.12945
	ea4eig	100	-77.4438	-69.5914	3%	3.34186
	jso	1152	-64.8424	-58.998	3%	2.20284
	lmcmaes	17	-79.5563	-72.7176	3%	4.43642
	mlshaderl	1800	-35.1957	-31.0584	3%	1.27761
	nlshadelbc	1151	-97.6209	-96.8704	3%	0.580958
	sfcde	1000	-34.8401	-33.2714	3%	0.845144
	sparq	100	-98.6917	-97.0668	3%	0.964969

Table 11. Effect of population-size fairness on the ten-problem comparison subset: SPARQ and the four competitors whose own recommended default already equals 100 (ARQ2, EA4Eig, TridentDE, UDE3) keep N=100; the remaining five competitors (jSO, LM-CMA-ES, mLSHADE-RL, NL-SHADE-LBC, SFCDE) are run with their own literature-recommended, dimension-dependent population size N.

PROBLEM	METHOD	N	BEST	MEAN	RATE%	SD
	tridentde	100	-65.0093	-59.4128	3%	2.37862
	ude3	100	-43.2703	-39.8999	3%	1.40427
rosenbrock [d=50]	arq2	100	4.29661	13.6938	3%	6.40239
	ea4eig	100	17.7119	21.4993	3%	1.87024
	jso	692	26.9656	29.1152	3%	1.13551
	lmcmaes	15	8.81167	12.4255	3%	2.48772
	mlshaderl	900	34.9874	36.9924	3%	0.782584
	nlshadelbc	692	44.8596	45.4138	3%	0.330367
	sfcde	500	42.5664	42.9479	3%	0.57513
	sparq	100	14.951	24.7064	3%	3.1607
	tridentde	100	35.9227	37.2389	3%	0.922804
	ude3	100	29.2749	31.35	3%	0.852551
rotatedrosenbrock [d=50]	arq2	100	0.00604507	18.4717	3%	17.4377
	ea4eig	100	17.1324	24.1876	3%	10.0064
	jso	692	28.0699	30.7051	3%	1.04521
	lmcmaes	15	1.9e-11	9.01549	3%	5.33793
	mlshaderl	900	36.5399	37.7887	3%	0.613311
	nlshadelbc	692	0.0796359	0.466404	3%	0.29581
	sfcde	500	41.1462	78.3944	3%	31.2707
	sparq	100	0.000444079	27.671	3%	15.0013
	tridentde	100	0.00370444	31.8263	3%	38.7057
	ude3	100	0.037282	43.6173	3%	33.4042
weierstrass [d=70]	arq2	100	0.0444884	3.11574	3%	2.21603
	ea4eig	100	0.00735326	0.2273	3%	0.251044
	jso	889	0.0379418	0.123304	3%	0.0751082
	lmcmaes	16	0	0	100%	0
	mlshaderl	1260	0.0350367	0.0624252	3%	0.0167006
	nlshadelbc	889	1.64097	2.78678	3%	0.555759
	sfcde	700	0.120609	0.639522	3%	0.412534
	sparq	100	0	0	100%	0
	tridentde	100	1.33079	5.02133	3%	2.10635
	ude3	100	0.166803	0.776516	3%	0.432902

849

Table 12 summarizes the outcome, reporting the mean rank under the mean-value criterion and the average success rate for each method across the ten problems. SPARQ retains the best mean rank, 3.55, and the highest average success rate, 43.9%, even after every eligible competitor is given its own recommended, and in most cases substantially larger, population. EA4Eig follows closely in mean rank, 3.65, but with a markedly lower average success rate, 22.8%, reflecting the same distinction between rank-based and success-rate-based comparison already observed in Section 4.1. Two competitors improve their own standing relative to the fixed-100 comparison, most visibly mLSHADE-RL, whose average success rate rises from its Section 4.1 figure once it is allowed its own NPinit=18D, and jSO, which likewise benefits from its larger recommended population on the higher-dimensional problems in this subset. Neither change alters the overall ordering at the top: SPARQ and EA4Eig remain the two best-ranked methods under either population convention.

Table 12. Mean rank (mean-value criterion) and average success rate of each method on the ten-problem population-fairness subset, with the population rule actually used.

Method	Population rule used	Mean rank (mean-value)	Avg. success rate (%)
sparq	$N = 100$ (unchanged control)	3.55	43.9
ea4eig	$N = 100$ (own default)	3.65	22.8
arq2	$N = 100$ (own default)	4.40	10.8
mlshaderl	$N = 18D$	5.25	37.5
jso	$N = ceil(25 * sqrt(D) * lnD)$	5.35	23.1
ude3	$N = 100$ (own default)	5.35	25.4
lmcmaes	$N = 4 + floor(3lnD)$	5.40	32.5
tridentde	$N = 100$ (own default)	6.70	18.4
nlshadelbc	$N = ceil(25 * sqrt(D) * lnD)$	6.85	7.8
sfcde	$N = 10D$	8.50	13.1

This experiment indicates that the common-population convention adopted in Section 4.1 does not by itself account for SPARQ's comparative advantage. Giving five of the nine competitors their own, often much larger, recommended population size changes their individual results, sometimes favourably, but does not change which method attains the best mean rank or the highest success rate on this subset. As with the ablation study, this check was restricted to the same ten-problem subset for computational tractability, and a full re-evaluation across the entire forty-five-problem benchmark under each method's own recommended settings remains outside the scope of the present study.

4.7. Sensitivity Analysis of Key Parameters

SPARQ exposes close to thirty configurable parameters (Table 1), far more than can be swept exhaustively, a sensitivity analysis therefore requires a principled way to select a small, defensible subset. We selected the four mechanisms identified in Section 4.5 as having the clearest, statistically significant contribution on the ablation subset, on the reasoning that a parameter governing a mechanism already shown to matter is more informative to test than one governing a mechanism whose overall contribution is itself uncertain: the external archive, controlled by the archive-rate parameter α_A (significant on 4 of 10 problems in Table 10), the auxiliary IDE branch, whose usage is governed by the Thompson-sampling bootstrap length n_{bs} and posterior decay rate γ_b (IDE was significant on 6 of 10 problems, the largest count of any mechanism tested), and the SHADE-style memory, controlled by the NLPSR shrink-shape exponent α_{NL} governing the population schedule alongside it. An initial sweep additionally included the SHADE memory size H, but this was withdrawn after the collected results showed byte-identical outcomes across all four tested values on every one of the ten problems, including problems on which the other parameters clearly do vary, this pattern is inconsistent with H genuinely having no

effect under a stochastic algorithm and points instead to a data-collection issue in that particular sweep, so it was replaced with the bandit decay rate γ_b as a second, independent dial on the IDE-branch mechanism. Each of the four retained parameters, α_A, n_{bs}, α_{NL}, and γ_b, was swept over four candidate values, including its Table 1 default, on the same ten-problem subset used for the ablation study, with every other parameter held at its default and 30 independent runs per value-problem combination.

Table 13. Sensitivity of SPARQ to four parameters on the ten-problem ablation subset (30 runs per value-problem combination; all other parameters at their Table 1 defaults; default value of each parameter shown in bold).

PARAMETER	PROBLEM	VALUE	MEAN	SD	MIN	MAX	MAIN EFFECT	RANK
alphaA	**alpine1 [d=100]**	0.5	2.95621e-09	6.44244e-08	0	1.41258e-06	4.46618e-09	2
		1	4.6639e-11	1.02101e-09	0	2.2387e-08	4.46618e-09	1
		1.5	**4.08727e-09**	**6.98328e-08**	**0**	**1.45131e-06**	**4.46618e-09**	**3**
		2.5	4.51282e-09	9.36744e-08	0	2.05091e-06	4.46618e-09	4
	rosenbrock [d=50]	0.5	30.8818	2.44486	22.4713	37.2821	8.53762	4
		1	26.5553	2.95205	17.7903	34.166	8.53762	3
		1.5	**24.4109**	**3.38834**	**12.6066**	**32.117**	**8.53762**	**2**
		2.5	22.3442	3.92339	10.3281	31.8682	8.53762	1
	rotatedrosenbrock [d=50]	0.5	31.6445	17.1922	5.30923e-05	79.3638	6.09867	4
		1	27.7785	15.9357	2.99216e-08	81.9867	6.09867	3
		1.5	**25.5458**	**14.4953**	**4.79067e-06**	**82.9222**	**6.09867**	**1**
		2.5	25.8389	12.1488	0.0124478	84.445	6.09867	2
	katsuura [d=50]	0.5	4.38266e-09	6.46704e-08	0	1.33446e-06	3.33005e-09	4
		1	1.05261e-09	2.17262e-08	0	4.75909e-07	3.33005e-09	1
		1.5	**1.07543e-09**	**2.05351e-08**	**0**	**4.48515e-07**	**3.33005e-09**	**2**
		2.5	2.01786e-09	3.56689e-08	0	7.56872e-07	3.33005e-09	3
	michalewicz [d=100]	0.5	-96.9699	1.24534	-98.9542	-91.3617	0.0643073	1
		1	-96.9056	1.23683	-98.9133	-90.659	0.0643073	4
		1.5	**-96.9545**	**1.22561**	**-99.1026**	**-91.4756**	**0.0643073**	**2**
		2.5	-96.9541	1.06503	-98.8759	-92.7143	0.0643073	3
	gallagher101 [d=16]	0.5	2.10022	1.66466	0	4.71372	0.377587	2
		1	2.36787	2.19535	0	14.2467	0.377587	3
		1.5	**2.37155**	**2.37385**	**0**	**14.2467**	**0.377587**	**4**
		2.5	1.99396	1.57924	0	4.71372	0.377587	1
	antennaarray [d=12]	0.5	0.00680964	2.89894e-09	0.00680964	0.0068097	8.29233e-05	1
		1	0.00680964	3.5091e-08	0.00680964	0.00681041	8.29233e-05	2
		1.5	**0.00680964**	**2.85157e-08**	**0.00680964**	**0.00681007**	**8.29233e-05**	**3**
		2.5	0.00689256	0.00135662	0.00680964	0.03358	8.29233e-05	4
	eld3 [d=15]	0.5	32454.3	59.6682	32375.3	32712.4	17.0472	4

Table 13. Sensitivity of SPARQ to four parameters on the ten-problem ablation subset (30 runs per value-problem combination; all other parameters at their Table 1 defaults; default value of each parameter shown in bold).

PARAMETER	PROBLEM	VALUE	MEAN	SD	MIN	MAX	MAIN EFFECT	RANK
		1	32443.4	55.2389	32367.6	32675.7	17.0472	2
		1.5	**32447.5**	**57.578**	**32375.3**	**32848**	**17.0472**	**3**
		2.5	32437.3	50.916	32367.6	32644.9	17.0472	1
	hydrothermal [d=120]	0.5	141682	12.3102	141659	141733	8.87414	4
		1	141676	11.1823	141656	141734	8.87414	3
		1.5	**141673**	**11.2709**	**141656**	**141731**	**8.87414**	**1**
		2.5	141674	13.5404	141656	141744	8.87414	2
	weierstrass [d=70]	0.5	0	n/a	0	0	0	1
		1	0	n/a	0	0	0	2
		1.5	**0**	**n/a**	**0**	**0**	**0**	**3**
		2.5	0	n/a	0	0	0	4
nbs	**alpine1 [d=100]**	0	7.22895e-09	1.13466e-07	0	2.05091e-06	7.18775e-09	4
		2	**4.11993e-11**	**9.01925e-10**	**0**	**1.9776e-08**	**7.18775e-09**	**1**
		5	4.09271e-09	6.98341e-08	0	1.45131e-06	7.18775e-09	3
		10	2.4008e-10	5.25578e-09	0	1.15239e-07	7.18775e-09	2
	rosenbrock [d=50]	0	26.2657	4.46691	10.3281	35.6352	0.475091	4
		2	**26.2339**	**4.43431**	**13.9996**	**37.2821**	**0.475091**	**3**
		5	25.9019	4.63933	11.1169	36.8478	0.475091	2
		10	25.7907	4.50538	12.6066	36.4015	0.475091	1
	rotatedrosenbrock [d=50]	0	28.0288	14.5015	0.0015193	84.445	2.95305	2
		2	**28.2143**	**15.3843**	**4.79067e-06**	**77.4062**	**2.95305**	**3**
		5	25.8058	15.6221	2.91692e-05	82.9222	2.95305	1
		10	28.7588	15.3233	2.99216e-08	81.9867	2.95305	4
	katsuura [d=50]	0	1.05244e-09	1.68426e-08	0	3.54472e-07	3.52307e-09	2
		2	**4.49552e-09**	**6.61282e-08**	**0**	**1.33446e-06**	**3.52307e-09**	**4**
		5	2.00816e-09	3.56665e-08	0	7.56872e-07	3.52307e-09	3
		10	9.72449e-10	2.04644e-08	0	4.48515e-07	3.52307e-09	1
	michalewicz [d=100]	0	-96.9423	1.19598	-98.9667	-91.9293	0.160434	3
		2	**-96.8647**	**1.20539**	**-98.8759**	**-92.3297**	**0.160434**	**4**
		5	-96.9518	1.23933	-99.1026	-91.3617	0.160434	2
		10	-97.0252	1.13453	-98.9045	-90.659	0.160434	1
	gallagher101 [d=16]	0	1.69774	1.41969	0	4.71372	0.785357	1
		2	**2.4831**	**2.22188**	**0**	**14.2467**	**0.785357**	**4**
		5	2.31389	1.9852	0	14.2467	0.785357	2
		10	2.33887	2.13815	0	14.2467	0.785357	3

Table 13. Sensitivity of SPARQ to four parameters on the ten-problem ablation subset (30 runs per value-problem combination; all other parameters at their Table 1 defaults; default value of each parameter shown in bold).

PARAMETER	PROBLEM	VALUE	MEAN	SD	MIN	MAX	MAIN EFFECT	RANK
	antennaarray [d=12]	0	0.00680964	4.11624e-08	0.00680964	0.00681025	5.57777e-05	1
		2	**0.00686542**	**0.00122094**	**0.00680964**	**0.03358**	**5.57777e-05**	**4**
		5	0.00683677	0.000593937	0.00680964	0.0198323	5.57777e-05	3
		10	0.00680964	7.2643e-08	0.00680964	0.0068109	5.57777e-05	2
	eld3 [d=15]	0	32448.3	57.4612	32367.6	32701.8	5.58767	4
		2	**32442.7**	**57.3835**	**32367.6**	**32848**	**5.58767**	**1**
		5	32443.9	54.8841	32367.6	32675.7	5.58767	2
		10	32447.6	55.1698	32375.3	32667.1	5.58767	3
	hydrothermal [d=120]	0	141676	13.2705	141656	141733	0.446012	4
		2	**141676**	**11.7193**	**141656**	**141734**	**0.446012**	**1**
		5	141676	12.5791	141656	141731	0.446012	3
		10	141676	12.8963	141656	141744	0.446012	2
	weierstrass [d=70]	0	0	n/a	0	0	0	1
		2	**0**	**n/a**	**0**	**0**	**0**	**2**
		5	0	n/a	0	0	0	3
		10	0	n/a	0	0	0	4
alphaNL	**alpine1 [d=100]**	0.2	0	0	0	0	1.02937e-08	1
		0.5	**0**	**0**	**0**	**0**	**1.02937e-08**	**2**
		0.8	1.30922e-09	2.30026e-08	0	4.90802e-07	1.02937e-08	3
		1	1.02937e-08	1.31195e-07	0	2.05091e-06	1.02937e-08	4
	rosenbrock [d=50]	0.2	28.0182	4.13349	14.2042	37.2821	3.49553	4
		0.5	**26.5127**	**4.39123**	**12.9785**	**35.1944**	**3.49553**	**3**
		0.8	25.1388	4.46483	11.1169	36.4015	3.49553	2
		1	24.5227	4.24851	10.3281	33.7528	3.49553	1
	rotatedrosenbrock [d=50]	0.2	29.3564	14.0555	0.000329929	48.4022	2.99601	4
		0.5	**28.0438**	**14.329**	**2.91692e-05**	**70.1072**	**2.99601**	**3**
		0.8	27.0472	15.7016	2.99216e-08	84.445	2.99601	2
		1	26.3604	16.6255	6.22551e-05	82.9222	2.99601	1
	katsuura [d=50]	0.2	0	0	0	0	7.5267e-09	1
		0.5	**0**	**0**	**0**	**0**	**7.5267e-09**	**2**
		0.8	1.00186e-09	2.05059e-08	0	4.48515e-07	7.5267e-09	3
		1	7.5267e-09	7.6783e-08	0	1.33446e-06	7.5267e-09	4
	michalewicz [d=100]	0.2	-96.9036	1.11853	-98.9133	-91.4756	0.0645788	4
		0.5	**-96.9682**	**1.23899**	**-99.1026**	**-90.659**	**0.0645788**	**1**
		0.8	-96.9675	1.21391	-98.9542	-91.3617	0.0645788	2

Table 13. Sensitivity of SPARQ to four parameters on the ten-problem ablation subset (30 runs per value-problem combination; all other parameters at their Table 1 defaults; default value of each parameter shown in bold).

PARAMETER	PROBLEM	VALUE	MEAN	SD	MIN	MAX	MAIN EFFECT	RANK
		1	-96.9447	1.20699	-98.8759	-91.6953	0.0645788	3
	gallagher101 [d=16]	0.2	2.21919	1.9606	0	14.2467	0.047313	3
		0.5	**2.20712**	**2.04543**	**0**	**14.2467**	**0.047313**	**2**
		0.8	2.17999	1.89946	0	14.2467	0.047313	1
		1	2.2273	2.04758	0	14.2467	0.047313	4
	antennaarray [d=12]	0.2	0.00680965	1.15286e-07	0.00680964	0.00681156	8.29056e-05	3
		0.5	**0.00680965**	**8.05359e-08**	**0.00680964**	**0.00681122**	**8.29056e-05**	**2**
		0.8	0.00680964	1.22382e-08	0.00680964	0.00680988	8.29056e-05	1
		1	0.00689254	0.00135662	0.00680964	0.03358	8.29056e-05	4
	eld3 [d=15]	0.2	32448.3	59.1472	32375.3	32848	5.11361	4
		0.5	**32445.1**	**53.821**	**32367.6**	**32649.9**	**5.11361**	**2**
		0.8	32445.8	58.4534	32367.6	32712.4	5.11361	3
		1	32443.2	53.3646	32367.6	32675.4	5.11361	1
	hydrothermal [d=120]	0.2	141680	13.5136	141656	141731	5.07982	4
		0.5	**141676**	**12.6256**	**141656**	**141744**	**5.07982**	**3**
		0.8	141675	11.97	141656	141734	5.07982	2
		1	141675	11.6667	141656	141721	5.07982	1
	weierstrass [d=70]	0.2	0	n/a	0	0	0	1
		0.5	**0**	**n/a**	**0**	**0**	**0**	**2**
		0.8	0	n/a	0	0	0	3
		1	0	n/a	0	0	0	4
gammab	**alpine1 [d=100]**	0.9	0	n/a	0	0	0	1
		0.95	0	n/a	0	0	0	2
		0.97	**0**	**n/a**	**0**	**0**	**0**	**3**
		0.99	0	n/a	0	0	0	4
	rosenbrock [d=50]	0.9	25.7995	3.66568	16.0684	31.3148	1.51732	4
		0.95	24.3949	3.48478	16.7474	30.3828	1.51732	2
		0.97	**24.7064**	**3.1607**	**14.951**	**32.1077**	**1.51732**	**3**
		0.99	24.2822	2.44077	18.9412	28.3387	1.51732	1
	rotatedrosenbrock [d=50]	0.9	33.9536	13.7696	3.87226	48.4808	6.50744	4
		0.95	30.2473	12.6282	0.385814	48.4929	6.50744	3
		0.97	**27.671**	**15.0013**	**0.000444079**	**48.7565**	**6.50744**	**2**
		0.99	27.4462	10.4656	0.0149223	48.6187	6.50744	1
	katsuura [d=50]	0.9	0	n/a	0	0	0	1
		0.95	0	n/a	0	0	0	2

Table 13. Sensitivity of SPARQ to four parameters on the ten-problem ablation subset (30 runs per value-problem combination; all other parameters at their Table 1 defaults; default value of each parameter shown in bold).

PARAMETER	PROBLEM	VALUE	MEAN	SD	MIN	MAX	MAIN EFFECT	RANK
		0.97	**0**	**n/a**	**0**	**0**	**0**	**3**
		0.99	0	n/a	0	0	0	4
	michalewicz [d=100]	0.9	-96.4399	1.13209	-98.4282	-92.8158	1.12419	4
		0.95	-97.0213	0.976817	-98.3477	-95.3237	1.12419	3
		0.97	**-97.0668**	**0.964969**	**-98.6917**	**-95.0356**	**1.12419**	**2**
		0.99	-97.5641	0.851981	-98.7876	-95.2536	1.12419	1
	gallagher101 [d=16]	0.9	0	n/a	0	0	0	1
		0.95	0	n/a	0	0	0	2
		0.97	**0**	**n/a**	**0**	**0**	**0**	**3**
		0.99	0	n/a	0	0	0	4
	antennaarray [d=12]	0.9	0.00680965	4.134e-08	0.00680964	0.00680986	8.466e-09	4
		0.95	0.00680964	4.15439e-09	0.00680964	0.00680966	8.466e-09	2
		0.97	**0.00680964**	**1.79729e-10**	**0.00680964**	**0.00680964**	**8.466e-09**	**1**
		0.99	0.00680964	1.39913e-08	0.00680964	0.00680971	8.466e-09	3
	eld3 [d=15]	0.9	32450.4	52.8995	32375.3	32628.4	20.536	2
		0.95	32450.6	52.1123	32375.3	32544.4	20.536	3
		0.97	**32440.5**	**47.9137**	**32375.3**	**32541**	**20.536**	**1**
		0.99	32461	91.2458	32375.3	32799.7	20.536	4
	hydrothermal [d=120]	0.9	141678	12.5879	141659	141716	8.347	4
		0.95	141671	9.69727	141657	141698	8.347	3
		0.97	**141669**	**9.8093**	**141656**	**141696**	**8.347**	**1**
		0.99	141670	11.5006	141656	141701	8.347	2
	weierstrass [d=70]	0.9	0	n/a	0	0	0	1
		0.95	0	n/a	0	0	0	2
		0.97	**0**	**n/a**	**0**	**0**	**0**	**3**
		0.99	0	n/a	0	0	0	4

Table 13 reports the mean, standard deviation, range, a one-way main-effect statistic, and the resulting rank (1=best) for each tested value of each parameter on each problem. For all four parameters, the main-effect statistic is non-zero, and therefore the ranking meaningful, on nine of the ten problems. The exception is consistent across all four: alpine1, katsuura, gallagher101, and weierstrass together account for every remaining zero-effect case, since SPARQ already reaches the exact global optimum on every run on these four problems regardless of the setting tested (Section 4.1), leaving no room for any parameter to make a measurable difference there.

Averaged across the ten problems, the archive rate α_A obtains nearly indistinguishable mean ranks for its four candidate values, 2.70, 2.40, 2.40, and 2.50 for 0.5, 1, 1.5, and 2.5 respectively, the Table 1 default, 1.5, ties for the best average rank with the smaller value of

1. The bootstrap length n_{bs} shows a mild preference for longer values, with mean ranks of 2.60, 2.70, 2.40, and 2.30 for 0, 2, 5, and 10 respectively, the default, 2, is in fact the worst-performing of the four values on average, ranking last on 4 of the 10 problems and best on only 3. The NLPSR shrink-shape exponent α_{NL} shows mean ranks of 2.90, 2.20, 2.20, and 2.70 for 0.2, 0.5, 0.8, and 1 respectively, the default, 0.5, ties for the best average rank with 0.8 and is never the worst. The bandit decay rate γ_b shows mean ranks of 2.60, 2.40, 2.20, and 2.80 for 0.9, 0.95, 0.97, and 0.99 respectively, the default, 0.97, obtains the best average rank of the four, is the single best-ranked choice on 3 of the 10 problems, and is never the worst-ranked choice on any of them.

Taken together, three of the four defaults, α_A, α_{NL}, and γ_b, are reasonably well chosen: each ties for, or outright obtains, the best average rank among the tested alternatives, and none is ever the single worst choice on any of the ten problems. The bootstrap length is the exception: its default value is, on average, the weakest of the four settings tested, suggesting that a longer forced-ARQ warm-up before the Thompson bandit takes over may be preferable to the current setting of 2 iterations. As with the ablation study, this sensitivity analysis was restricted to the same ten-problem subset for computational tractability, and a full sweep across the entire benchmark suite, together with a corrected re-run for the SHADE memory size H, remains outside the scope of the present study.

5. Conclusions

This work introduced SPARQ, a differential-evolution-based optimizer that extends the ARQ2 architecture by replacing nearly every one of its fixed or singly-adapted mechanisms with an adaptive counterpart, and by adding several mechanisms with no counterpart in its predecessor, including a stagnation-gated elite polish phase, a trajectory-echo memory, and a Solis-Wets local search probe. The experimental evaluation covered both standard-dimensionality and large-scale problem sets, comprising a combined total of seventy-nine problem instances and spanning synthetic test functions alongside real-world engineering formulations.

Across the standard-dimensionality benchmark, SPARQ achieved the highest average success rate and the lowest mean rank among ten competing methods under the best-value criterion, and a closely contested second-lowest mean rank, narrowly behind EA4Eig, under the stricter mean-value criterion. Its advantage was most pronounced on problems where competing methods were confined to low single-digit success rates, while on real-world problems with intrinsically difficult landscapes, such as the economic load dispatch family and several chemical-engineering formulations, no method in the comparison, SPARQ included, achieved consistently high success rates, indicating that the difficulty in these cases is a property of the problem rather than of any particular optimizer. A similar pattern held on the large-scale benchmark, where SPARQ was compared against five optimizers developed specifically for high-dimensional search: it attained the highest success rate and the lowest mean rank of the six methods, with the one clear exception being the vincent function, on which two competing methods reached the exact optimum more reliably.

The results support the central premise of this work, namely that extending nearly every component of ARQ2's architecture with an adaptive counterpart yields a method that performs competitively on both real-world and synthetic problems, and at both standard and large-scale dimensionality, without being restricted to a narrow class of problem structures. A further observation is that this breadth of adaptivity does not come at a disproportionate computational cost: despite the substantially larger number of mechanisms it coordinates relative to its predecessor and to several competing methods, SPARQ's execution times remained among the lowest recorded in both parts of the study, rather than scaling upward with the added algorithmic complexity. This premise is further supported by the population-size fairness check of Section 4.6: when five of the nine standard-dimensionality competitors are reconfigured to their own literature-recommended, and in most cases substantially larger, population sizes, SPARQ retains both the best mean rank and the highest average success rate on the ten-problem subset tested, indicating that its

advantage is not an artifact of the common population-size convention adopted elsewhere in the study.

At the same time, the results also delineate the boundaries of this approach. SPARQ's reliability advantage narrows or disappears on a specific subset of problems, most notably vincent among the large-scale functions and several real-world load-dispatch and material-potential problems among the standard-dimensionality set, suggesting that the current balance of its exploration and exploitation mechanisms is better suited to some landscape types than others. The ablation results of Section 4.5 reinforce this picture: several individual mechanisms show no statistically significant contribution on the tested subset, and the full method does not always obtain the best mean rank among its own leave-one-out variants, indicating that SPARQ's robustness should be understood as an aggregate property of the full configuration rather than a guaranteed contribution from every constituent mechanism.

Several directions follow naturally from the present findings. First, the subset of problems on which SPARQ's advantage narrows, particularly vincent and the economic load dispatch family, warrants closer examination to determine whether their difficulty stems from deceptive multimodality, ill-conditioning, or a mismatch with the current elite-polish and quarantine mechanisms, and whether a targeted adjustment could close this gap without weakening performance elsewhere. Second, a first leave-one-out ablation analysis is now reported in Section 4.5 on a representative ten-problem subset, extending it to the full benchmark suite would help confirm which mechanisms drive the bulk of the observed improvement over ARQ2 across a wider range of landscapes, informing a leaner variant of the algorithm. Third, the present evaluation was limited to box-constrained, single-objective continuous problems, extending SPARQ to handle general nonlinear constraints, multiple objectives, or mixed-integer variables would broaden its applicability to a wider range of engineering design problems. Finally, given that several of SPARQ's mechanisms, such as the SHADE-style memory and the Thompson-sampling bandit, involve internally tunable hyperparameters, an empirical sensitivity study of these settings across problem classes would clarify the robustness of the default configuration used in this work.

1. Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. IEEE Transactions on Evolutionary Computation, 1(1), 67–82. DOI: 10.1109/4235.585893
2. Solis, F. J., & Wets, R. J.-B. (1981). Minimization by random search techniques. Mathematics of Operations Research, 6(1), 19–30. DOI: 10.1287/moor.6.1.19
3. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. DOI: 10.1126/science.220.4598.671
4. Holland, J. H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.
5. Kyrou, G., Charilogis, V., & Tsoulos, I. G. (2024). Improving the Giant-Armadillo Optimization Method. Analytics, 3(2), 225–240. DOI: 10.3390/analytics3020013
6. Alsayyed, O., Hamadneh, T., Al-Tarawneh, H., Alqudah, M., Gochhait, S., Leonova, I., Malik, O. P., & Dehghani, M. (2023). Giant Armadillo Optimization: A New Bio-Inspired Metaheuristic Algorithm for Solving Optimization Problems. Biomimetics, 8(8), 619. DOI: 10.3390/biomimetics8080619
7. Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. Proceedings of the IEEE International Conference on Neural Networks, 4, 1942–1948. DOI: 10.1109/ICNN.1995.488968
8. Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. Evolutionary Computation, 9(2), 159–195. DOI: 10.1162/106365601750190398
9. Loshchilov, I. (2014). A computationally efficient limited memory CMA-ES for large scale optimization. Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), 397–404. DOI: 10.1145/2576768.2598294
10. Charilogis, V., Tsoulos, I. G., & Stavrou, V. N. (2023). An Intelligent Technique for Initial Distribution of Genetic Algorithms. Axioms, 12(10), 980. DOI: 10.3390/axioms12100980
11. Poikolainen, I., Neri, F., & Caraffini, F. (2015). Cluster-based population initialization for differential evolution frameworks. Information Sciences, 297, 216–235. DOI: 10.1016/j.ins.2014.11.026
12. Storn, R., & Price, K. (1997). Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization, 11(4), 341–359. DOI: 10.1023/A:1008202821328

13. Das, S., & Suganthan, P. N. (2011). Differential evolution: A survey of the state-of-the-art. IEEE Transactions on Evolutionary Computation, 15(1), 4–31. DOI: 10.1109/TEVC.2010.2059031
14. Zhang, J., & Sanderson, A. C. (2009). JADE: Adaptive differential evolution with optional external archive. IEEE Transactions on Evolutionary Computation, 13(5), 945–958. DOI: 10.1109/TEVC.2009.2014613
15. Tanabe, R., & Fukunaga, A. (2013). Success-history based parameter adaptation for differential evolution. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 71–78. DOI: 10.1109/CEC.2013.6557555
16. Tanabe, R., & Fukunaga, A. S. (2014). Improving the search performance of SHADE using linear population size reduction. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 1658–1665. DOI: 10.1109/CEC.2014.6900380
17. Brest, J., & Maučec, M. S. (2008). Population size reduction for the differential evolution algorithm. Applied Intelligence, 29(3), 228–247.
18. Brest, J., Maučec, M. S., & Bošković, B. (2017). Single objective real-parameter optimization: Algorithm jSO. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 1311–1318. DOI: 10.1109/CEC.2017.7969456
19. Stanovov, V., Akhmedova, S., & Semenkin, E. (2018). LSHADE Algorithm with Rank-Based Selective Pressure Strategy for Solving CEC 2017 Benchmark Problems. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 1–8. DOI: 10.1109/CEC.2018.8477977
20. Stanovov, V., Akhmedova, S., & Semenkin, E. (2021). NL-SHADE-RSP algorithm with adaptive archive and selective pressure for CEC 2021 numerical optimization. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 809–816. DOI: 10.1109/CEC45853.2021.9504959
21. Wang, Y., Li, H.-X., Huang, T., & Li, L. (2014). Differential evolution based on covariance matrix learning and bimodal distribution parameter setting. Applied Soft Computing, 18, 232–247.
22. Bujok, P., & Tvrdík, J. (2017). Enhanced individual-dependent differential evolution with population size adaptation. Proceedings of the IEEE Congress on Evolutionary Computation (CEC).
23. Guo, S.-M., Tsai, J.-H., Yang, C.-C., & Hsu, P.-H. (2015). A self-optimization approach for L-SHADE incorporated with eigenvector-based crossover and successful-parent-selecting framework on CEC 2015 benchmark set. Proceedings of the IEEE Congress on Evolutionary Computation (CEC).
24. Bujok, P., & Kolenovský, P. (2022). Eigen crossover in cooperative model of evolutionary algorithms applied to CEC 2022 single objective numerical optimisation. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 1–8. DOI: 10.1109/CEC55065.2022.9870433
25. Tizhoosh, H. R. (2005). Opposition-based learning: A new scheme for machine intelligence. Proceedings of the International Conference on Computational Intelligence for Modelling, Control and Automation (CIMCA), 695–701. DOI: 10.1109/CIMCA.2005.1631345
26. Rahnamayan, S., Tizhoosh, H. R., & Salama, M. M. A. (2008). Opposition-based differential evolution. IEEE Transactions on Evolutionary Computation, 12(1), 64–79. DOI: 10.1109/TEVC.2007.894200
27. Mantegna, R. N. (1994). Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. Physical Review E, 49(5), 4677–4683. DOI: 10.1103/PhysRevE.49.4677
28. Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika, 25(3/4), 285–294. DOI: 10.1093/biomet/25.3-4.285
29. Charilogis, V., Tsoulos, I. G., Gianni, A. M., & Tsalikakis, D. (2025). ARQ: A cohesive optimization design for stable performance on noisy landscapes. Applied Sciences, 15(22), 12180. DOI: 10.3390/app152212180
30. Charilogis, V., Tsoulos, I. G., & Gianni, A. M. (2026). ARQ2: Toward stability-aware hybrid optimization on complex and noisy search problems. Symmetry, 18(5), 844. DOI: 10.3390/sym18050844
31. Charilogis, V., Tsoulos, I. G., & Gianni, A. M. (2026). OptimSolution: A Cross-Platform Framework for Benchmarking, Sensitivity, and Complexity Analysis of Continuous Optimisation Methods. Software, 5(3), 28. DOI: 10.3390/software5030028
32. Elsayed, S. M., Sarker, R. A., & Essam, D. L. (2011). Differential evolution with multiple strategies for solving CEC2011 real-world numerical optimization problems. In 2011 IEEE Congress on Evolutionary Computation (pp. 1041-1048). IEEE. https://doi.org/10.1109/CEC.2011.5949732
33. Chauhan, D., Trivedi, A., & Shivani. (2024). A Multi-operator Ensemble LSHADE with Restart and Local Search Mechanisms for Single-objective Optimization. arXiv:2409.15994. DOI: 10.48550/arXiv.2409.15994
34. Stanovov, V., Akhmedova, S., & Semenkin, E. (2022). NL-SHADE-LBC algorithm with linear parameter adaptation bias change for CEC 2022 Numerical Optimization. Proceedings of the IEEE Congress on Evolutionary Computation (CEC), 1–8. DOI: 10.1109/CEC55065.2022.9870295
35. Wang, Z., Li, D., Yu, J., Abdel-Salam, M., Hu, G., Houssein, E. H., Abdel Samee, N., & Zhong, R. (2026). Competitive differential evolution with success-failure adaptation mechanism: performance benchmarking and application in eggplant disease diagnosis. International Journal of Machine Learning and Cybernetics, 17, 155. DOI: 10.1007/s13042-025-02931-3
36. Charilogis, V., Tsoulos, I. G., & Gianni, A. M. (2025). TRIDENT-DE: Triple-Operator Differential Evolution with Adaptive Restarts and Greedy Refinement. Future Internet, 17(11), 488. DOI: 10.3390/fi17110488
37. Trivedi, A., & Chauhan, D. (2024). UDE-III: An Enhanced Unified Differential Evolution Algorithm for Constrained Optimization Problems. arXiv:2410.03992. DOI: 10.48550/arXiv.2410.03992

38. Das, S., & Suganthan, P. N. (2011). Problem Definitions and Evaluation Criteria for CEC 2011 Competition on Testing Evolutionary Algorithms on Real World Optimization Problems. Technical Report, Jadavpur University, Kolkata, India, and Nanyang Technological University, Singapore.
39. Thanedar, P. B., & Vanderplaats, G. N. (1995). Survey of discrete variable optimization for structural design. Journal of Structural Engineering, 121(2), 301–305. DOI: 10.1061/(ASCE)0733-9445(1995)121:2(301)
40. Charilogis, V., Tsoulos, I. G., Gianni, A. M., & Tsalikakis, D. (2026). A Novel Weather-Aware Irrigation Scheduling Benchmark for Continuous Global Optimization. Mathematics, 14(11), 2018. DOI: 10.3390/math14112018
41. Omidvar, M. N., Li, X., Mei, Y., & Yao, X. (2014). Cooperative Co-evolution with Differential Grouping for Large Scale Optimization. IEEE Transactions on Evolutionary Computation, 18(3), 378–393. DOI: 10.1109/TEVC.2013.2281543
42. Omidvar, M. N., Yang, M., Mei, Y., Li, X., & Yao, X. (2017). DG2: A Faster and More Accurate Differential Grouping for Large-Scale Black-Box Optimization. IEEE Transactions on Evolutionary Computation, 21(6), 929–942. DOI: 10.1109/TEVC.2017.2694221
43. He, X., Zheng, Z., & Zhou, Y. (2021). MMES: Mixture Model based Evolution Strategy for Large-Scale Optimization. IEEE Transactions on Evolutionary Computation, 25(2), 320–333. DOI: 10.1109/TEVC.2020.3034769
44. Li, Z., & Zhang, Q. (2018). A Simple Yet Efficient Evolution Strategy for Large-Scale Black-Box Optimization. IEEE Transactions on Evolutionary Computation, 22(5), 637–646. DOI: 10.1109/TEVC.2017.2765682
45. Akimoto, Y., & Hansen, N. (2016). Projection-based restricted covariance matrix adaptation for high dimension. Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), 197–204. DOI: 10.1145/2908812.2908863